CSA47 - Deep Learning · Assignment 2 (CO2)

Deep Feedforward and Convolutional Networks: Predict, Design, Implement, Measure, Diagnose, Redesign

Name: Harini. R

Register no: 192324108

Dataset and Setup

- Framework: PyTorch 2.15 (CPU only). Q2 is pure NumPy with no autograd. Python 3.13.
- Q1 and Q7: credit-card fraud (ULB, real data): all 492 frauds + 30,000 legitimate transactions, 30 features, stratified 60/20/20 split (18,295 train / 6,098 validation / 6,099 test).
- Q2: synthetic hospital-readmission cohort (4 features), 3,000 patients: 2,100 / 450 / 450.
- Q3: Fashion-MNIST used instead of leaf images. Training sets of 20,000 / 300 / 3,000 (30% label noise) images, 1,000 clean validation images, official 10,000 test images.
- Q4: synthetic hourly electricity load (3 years, 13 features), chronological split 17,353 / 4,368 / 4,391.
- Q5: Fashion-MNIST instead of PCB images: 20,000 / 5,000 / 10,000. Q6: Fashion-MNIST resized to 16×16 or 32×32: 10,000 / 5,000 / 10,000.
- Seeds 0, 1, 2 for every stochastic experiment. All values are the mean of 3 runs and are rounded to 3–4 decimals.
- Predictions were written before running (results/predictions_before_running.md, results/q7_prediction_before_running.md). Wrong predictions are reported.
- Synthetic data (Q2, Q4) and Fashion-MNIST (Q3, Q5, Q6) are stand-ins because real hospital, load, leaf, PCB and wildlife data were not reachable. The conclusions are about training behaviour, not about those industries.
- Code, logs and outputs are given in the Appendix and in the zip package.

Q1 – Hidden Units: Does the Activation Function Matter as the Network Gets Deeper?

1.1 Problem and setup

Bank fraud screening with a feedforward network (width 64, 30 inputs, 1 logit). Fraud is rare (1.6%), so the loss is binary cross-entropy with pos_weight = 10, and precision, recall and AUPRC are reported with accuracy (always predicting “legitimate” already gives about 98.4% accuracy). Training: SGD with momentum 0.9, lr 0.01, batch 256, 15 epochs.

Configuration	Hidden layers × width	Initialisation
Shallow	3 × 64	Xavier (sigmoid, tanh), He (ReLU)
Deep (≥ 8 layers)	12 × 64	Xavier / He
Edge case	12 × 64	N(0, 1) for every weight

1.2 Prediction before training

Error signal of layer l: $\delta_l = (W_{l+1}^T \delta_{l+1}) \odot \varphi'(z_l)$. Each layer multiplies the signal by about $\sigma_w\sqrt{n} \cdot \mathrm{rms}(\varphi')$. Xavier gives $\sigma_w\sqrt{n} = 1$, so the gain is set by φ'.

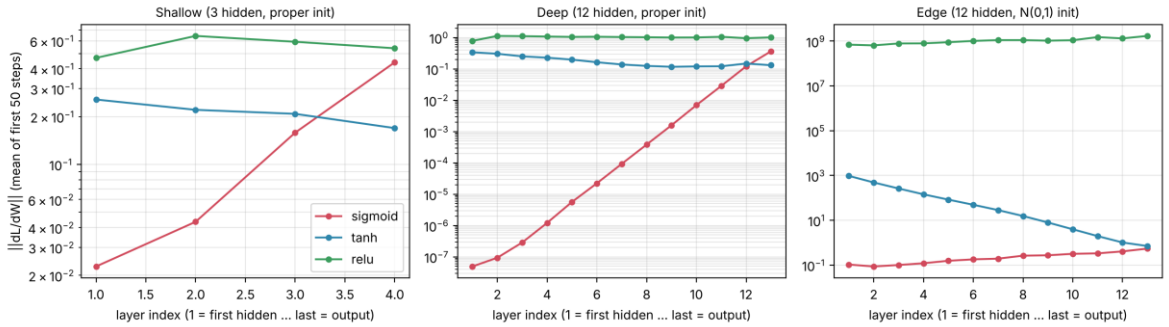
Activation	φ'	Predicted gain / layer	Predicted $\|g_1\| / \|g_{out}\|$ at L = 12
Sigmoid	$\sigma(1-\sigma) \le 0.25$	≈ 0.25 (vanishing)	$\approx 0.25^{11}$ = 2.4e-7, does not train
Tanh	$1 - \tanh^2 \le 1$	0.7 – 0.95	mild decay, trains slower than ReLU
ReLU (He)	0 or 1	≈ 1	flat across depth

Edge case (N(0,1) weights, $\sigma_w\sqrt{n} = 8$): I predicted that tanh and sigmoid saturate and their gradient vanishes, while ReLU explodes (gain ≈ 5.7 per layer).

1.3 Results

Layer-wise gradient norms during the first 50 updates (mean of 3 seeds):

Configuration	Act.	$\|g_1\|$	$\|g_{out}\|$	$\|g_1\|/\|g_{out}\|$	mean φ'	Dead ReLU init → end
Shallow (3 hidden)	sigmoid	2.27e-02	4.39e-01	5.12e-02	0.232	n/a
Shallow (3 hidden)	tanh	2.56e-01	1.69e-01	1.52e+00	0.827	n/a
Shallow (3 hidden)	relu	4.69e-01	5.39e-01	8.66e-01	0.479	0.0% → 0.0%
Deep (12 hidden)	**sigmoid**	**4.91e-08**	**3.65e-01**	**1.29e-07**	**0.235**	**n/a**
Deep (12 hidden)	**tanh**	**3.39e-01**	**1.31e-01**	**2.59e+00**	**0.914**	**n/a**
Deep (12 hidden)	**relu**	**7.84e-01**	**1.01e+00**	**7.96e-01**	**0.502**	**5.2% → 2.8%**
Edge (12, N(0,1) init)	sigmoid	1.06e-01	5.44e-01	1.79e-01	0.074	n/a
Edge (12, N(0,1) init)	tanh	9.60e+02	7.07e-01	1.33e+03	0.113	n/a
Edge (12, N(0,1) init)	relu	diverged	diverged	-	0.502	5.2% → 67.3%



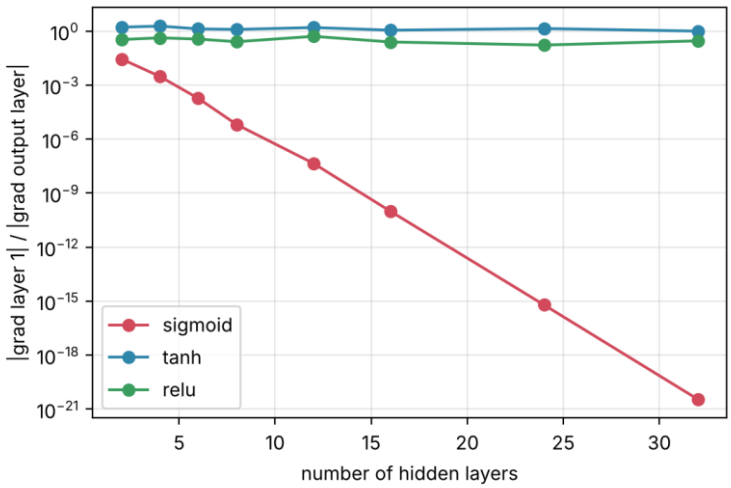
Performance after 15 epochs (validation set, threshold 0.5, 98 frauds):

Configuration	Act.	Train loss	Acc.	Prec.	Recall	AUPRC	Diverged
Shallow (3 hidden)	sigmoid	0.0999 ± 0.0013	0.995	0.829	0.881 ± 0.006	0.912	0/3
Shallow (3 hidden)	tanh	0.0539 ± 0.0037	0.993	0.724	0.915 ± 0.016	0.909	0/3
Shallow (3 hidden)	relu	0.0278 ± 0.0020	0.994	0.806	0.867 ± 0.020	0.886	0/3
Deep (12 hidden)	**sigmoid**	**0.4661 ± 0.0002**	**0.984**	**0.000**	**0.000 ± 0.000**	**0.068**	**0/3**
Deep (12 hidden)	**tanh**	**0.0391 ± 0.0048**	**0.992**	**0.734**	**0.867 ± 0.010**	**0.883**	**0/3**
Deep (12 hidden)	**relu**	**0.0265 ± 0.0121**	**0.990**	**0.638**	**0.881 ± 0.042**	**0.881**	**0/3**
Edge (12, N(0,1) init)	sigmoid	0.0695 ± 0.0049	0.990	0.674	0.786 ± 0.027	0.810	0/3

Configuration	Act.	Train loss	Acc.	Prec.	Recall	AUPRC	Diverged
Edge (12, N(0,1) init)	tanh	0.3060 ± 0.0335	0.925	0.167	0.687 ± 0.072	0.179	0/3
Edge (12, N(0,1) init)	relu	diverged (NaN)	0.984	0	0	0.016	3/3

Depth sweep at initialisation ($\|g_1\|/\|g_out\|$):

Hidden layers	Sigmoid + Xavier	Tanh + Xavier	ReLU + He
2	2.85e-02	1.66e+00	3.41e-01
4	3.05e-03	1.90e+00	4.25e-01
6	1.84e-04	1.33e+00	3.61e-01
8	6.48e-06	1.24e+00	2.59e-01
12	4.48e-08	1.59e+00	5.14e-01
16	9.83e-11	1.13e+00	2.50e-01
24	6.15e-16	1.39e+00	1.69e-01
32	3.54e-21	1.00e+00	2.89e-01



- **Deep sigmoid:** ratio 1.29e-7 (predicted 2.4e-7), decay 0.252 per layer, mean $\sigma' = 0.235$. The units are not saturated; the 0.25 ceiling of σ' itself removes the signal. Training loss stays at 0.466 (first epoch 0.473) and validation recall is 0 in all 3 seeds. At 32 layers the ratio is 3.54e-21.
- **Deep tanh and ReLU:** ratios 2.59 and 0.80. The sweep stays between 1.0 and 1.9 (tanh) and 0.17 and 0.51 (ReLU) up to 32 layers. Training loss 0.0391 (tanh), 0.0265 (ReLU). Dead ReLU units: 5.2% at init, 2.8% at the end.
- **Edge case:** ReLU diverges to NaN in 3/3 seeds (first-layer gradient 2e+8 – 1e+9). Tanh explodes towards the input ($\|g_1\| = 9.6e+2$ vs $\|g_out\| = 0.71$) and trains badly (AUPRC 0.18). Sigmoid still trains (recall 0.79, mean $\sigma' = 0.074$); a plausible reason, not tested separately, is that the large weights compensate for the small σ'.

1.4 Predicted vs observed

Prediction	Observed	Verdict
Deep sigmoid: 0.25× per layer, does not train	ratio 1.29e-7, recall 0	Correct

Prediction	Observed	Verdict
Deep tanh: mild decay 0.7 – 0.95	no decay, ratio 2.59	Partly wrong (better, because $\tanh'(0) = 1$)
Deep ReLU + He: flat gradient	ratio 0.80, flat to 32 layers	Correct
Shallow: all train	recall 0.88 / 0.91 / 0.87	Correct
Edge: ReLU explodes	NaN in 3/3 seeds	Correct
Edge: tanh and sigmoid vanish	tanh explodes, sigmoid trains	Wrong: I used the mean φ' instead of $\mathrm{rms}(\varphi')$ in the gain (likely cause)

1.5 Design decision (at least 8 hidden layers)

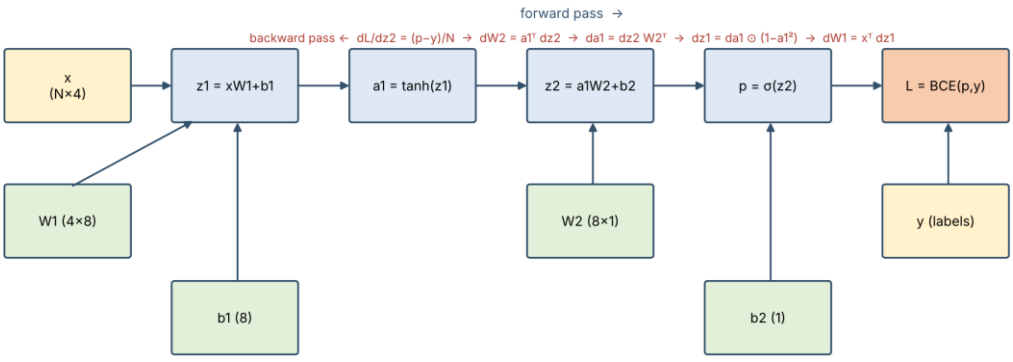
- **Choose ReLU with He initialisation** (12 layers). Tanh with Xavier is an acceptable alternative. Never sigmoid, never N(0,1) weights.
- Evidence: sigmoid fails (ratio 1.29e-7, recall 0). ReLU + He keeps the gradient flat from 2 to 32 layers and reached the lowest training loss (0.0265 vs 0.0391 tanh). The same ReLU network diverged with N(0,1) weights, so initialisation matters as much as the activation.
- Caveat: validation AUPRC of ReLU (0.88) and tanh (0.88) is equal within noise, and ReLU precision at threshold 0.5 is lower (0.64 vs 0.73). With 98 validation frauds a few false positives move precision by several points, so tune the threshold on a larger set. Monitor dead units (2–5% here).

Q2 – Can You Trust Your Gradients? Backpropagation from First Principles

2.1 Problem and model

30-day readmission from four features (age, length of stay, prior admissions, number of medications). Synthetic cohort of 3,000 patients (2,100 / 450 / 450, readmission rate 30.7%) generated from a logistic rule with an interaction term. Network: 4 → Dense(8, tanh) → Dense(1, sigmoid), mean binary cross-entropy, NumPy only. tanh is used because the ReLU kink at 0 breaks finite differences. Batch gradient descent, lr 0.5, 400 epochs, seed 0.

2.2 Computational graph



2.3 Chain-rule derivation

Forward: $z_1 = XW_1 + b_1$, $a_1 = \tanh(z_1)$, $z_2 = a_1W_2 + b_2$, $p = \sigma(z_2)$
Loss: $L = -(1/N)\ \Sigma\ [y \ln p + (1-y) \ln(1-p)]$

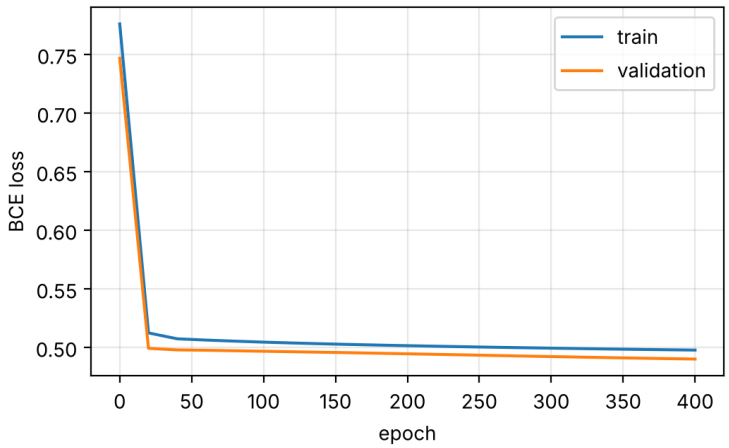
Step 1: $\partial L/\partial p = -(y/p - (1-y)/(1-p)) / N$ and $\sigma'(z_2) = p(1-p)$
Step 2: $\partial L/\partial z_2 = (p - y) / N$ (sigmoid and cross-entropy cancel)
Step 3: $\partial L/\partial W_2 = a_1^T(\partial L/\partial z_2)$, $\partial L/\partial b_2 = \Sigma(\partial L/\partial z_2)$
Step 4: $\partial L/\partial a_1 = (\partial L/\partial z_2)W_2^T$
Step 5: $\partial L/\partial z_1 = (\partial L/\partial a_1) \odot (1 - a_1^2)$ (tanh derivative)
Step 6: $\partial L/\partial W_1 = X^T(\partial L/\partial z_1)$, $\partial L/\partial b_1 = \Sigma(\partial L/\partial z_1)$

Shape check: (N×1)(1×8) = N×8; (4×N)(N×8) = 4×8; (8×N)(N×1) = 8×1.

2.4 Implementation

```
dz2 = (p − y) / N;  dW2 = a1.T @ dz2;  db2 = dz2.sum(0)
da1 = dz2 @ W2.T;  dz1 = da1 * (1 − a1**2)
dW1 = X.T @ dz1;  db1 = dz1.sum(0)
```

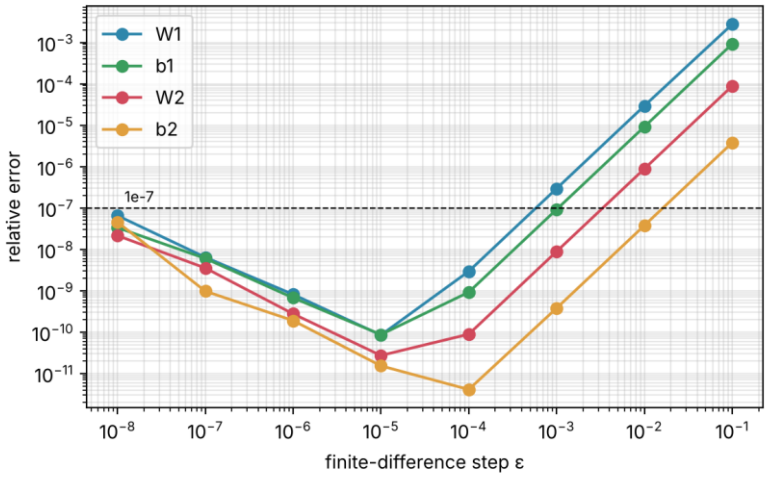
After training: trained 400 epochs lr=0.5: train loss 0.4977 val loss 0.4900 val acc 0.7911 test acc 0.7778 (majority baseline 0.7267). The network learns, so the analytic gradients are used correctly in the update.



2.5 Numerical gradient check

Central difference $(L(\theta+\varepsilon) - L(\theta-\varepsilon)) / 2\varepsilon$ for every scalar parameter, float64, 200 samples, random initialisation. Relative error = $||g_a - g_n|| / (||g_a|| + ||g_n||)$.

ε	W_1	b_1	W_2	b_2
1e-01	2.75e-03	9.20e-04	8.86e-05	3.78e-06
1e-02	2.94e-05	9.21e-06	8.87e-07	3.78e-08
1e-03	2.94e-07	9.21e-08	8.87e-09	3.79e-10
1e-04	2.95e-09	9.22e-10	8.90e-11	4.10e-12
1e-05	8.49e-11	8.72e-11	2.73e-11	1.53e-11
1e-06	8.16e-10	6.73e-10	2.77e-10	1.90e-10
1e-07	6.44e-09	6.02e-09	3.54e-09	9.75e-10
1e-08	6.55e-08	3.35e-08	2.15e-08	4.56e-08



- The curve is U-shaped: truncation error $O(\varepsilon^2)$ dominates for large ε (2.8e-3 at $\varepsilon = 0.1$), round-off error $O(u/\varepsilon)$ dominates for tiny ε (6.6e-8 at ε = 1e-8).
- Best ε = 1e-5: errors 8.49e-11, 8.72e-11, 2.73e-11, 1.53e-11 for (W_1, b_1, W_2, b_2). At the trained parameters: 1.99e-10, 6.01e-10, 8.37e-11, 1.47e-10.
- **Acceptance threshold: relative error < 1e-7** (float64, $\varepsilon \approx$ 1e-5). The achievable error is about 1e-10, and a real mistake gives 1e-1 to 1. Errors between 1e-7 and 1e-5 need a re-check with another ε; errors above 1e-3 mean the backward pass is wrong. For float32 or ReLU networks relax the threshold (about 1e-3 – 1e-2).

2.6 Deliberate bugs (ε = 1e-5)

Bug	W_1	b_1	W_2	b_2
missing_1_over_N	9.90e-01	9.90e-01	9.90e-01	9.90e-01
no_tanh_derivative	2.88e-01	2.06e-01	2.73e-11	1.53e-11
transpose_W2	6.56e-01	6.39e-01	2.73e-11	1.53e-11

- **no_tanh_derivative:** the factor $(1 - a_1^2)$ is forgotten. **transpose_W2:** W_2 is applied in the wrong order. **missing_1_over_N:** the 1/N of the mean is forgotten.
- For both hidden-layer bugs W_2 and b_2 still pass (2.7e-11, 1.5e-11) because $\partial L/\partial z_2$ is computed before the bug. **W_1 shows the largest error** (0.29, 0.66), then b_1 (0.21, 0.64). The first group that fails, moving from the output backwards, locates the bug between $\partial L/\partial z_2$ and $\partial L/\partial z_1$.
- A bug that fails all four groups equally (missing 1/N, error 0.99) is in the shared loss gradient at the start of the backward pass.

Q3 – Engineer Overfitting, Then Control It: Regularization

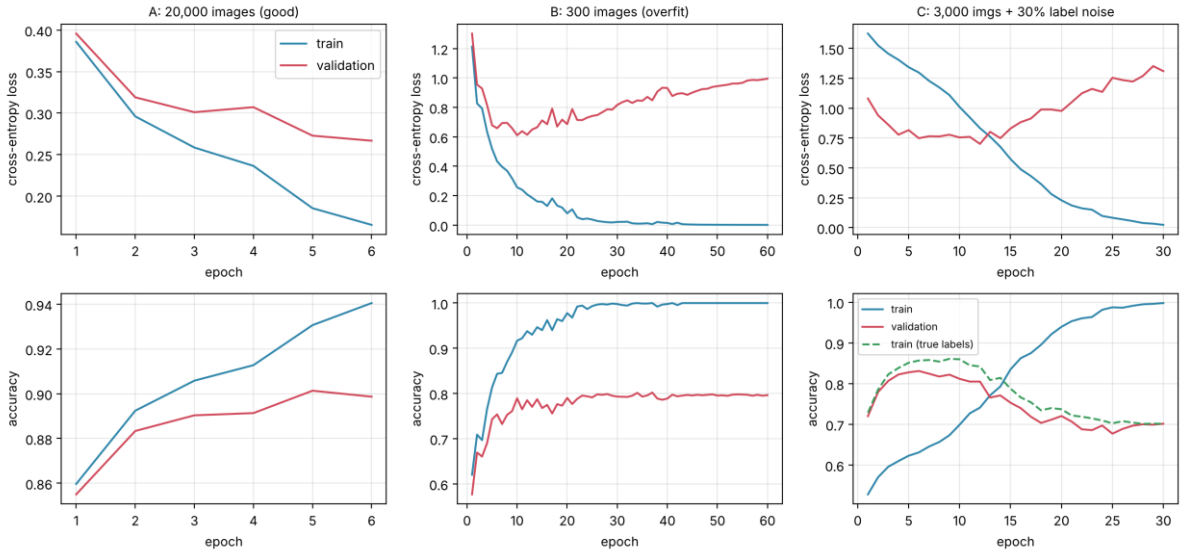
3.1 Setup

Plant-leaf images are not reachable, so Fashion-MNIST is used (10 classes, 28×28). Model: conv3×3(1→32) – ReLU – pool – conv3×3(32→64) – ReLU – pool – FC(3136→128) – ReLU – FC(128→10), about 0.42 M parameters. Adam, lr 1e-3, 3 seeds. Validation: 1,000 clean images disjoint from training. Test: official 10,000 images.

3.2 Three conditions: prediction before training and result

Condition	Predicted gap and reason	Train acc.	Val. acc.	Gap
A: 20,000 images, 6 epochs	3–6 points: n ≫ effective capacity, low variance	0.941 ± 0.003	0.899 ± 0.006	0.042 ± 0.004
B: 300 images, 60 epochs	25–35 points (train ≈ 100%, val 65–72%): 0.42 M parameters vs 300 samples (≈ 1,400 per sample) can memorise, high variance	1.000 ± 0.000	0.796 ± 0.013	0.204 ± 0.013
C: 3,000 images, 30% label noise, 30 epochs	20–30 points late in training: clean signal first, then wrong labels memorised	0.998 ± 0.001	0.701 ± 0.008	0.297 ± 0.007

Reason: test error = bias² + variance + noise. The model has low bias in all three conditions, so what changes is variance. More data constrains the parameters and keeps the gap small; 300 images let the network interpolate the training set; noisy labels are memorised after the clean pattern is learned.



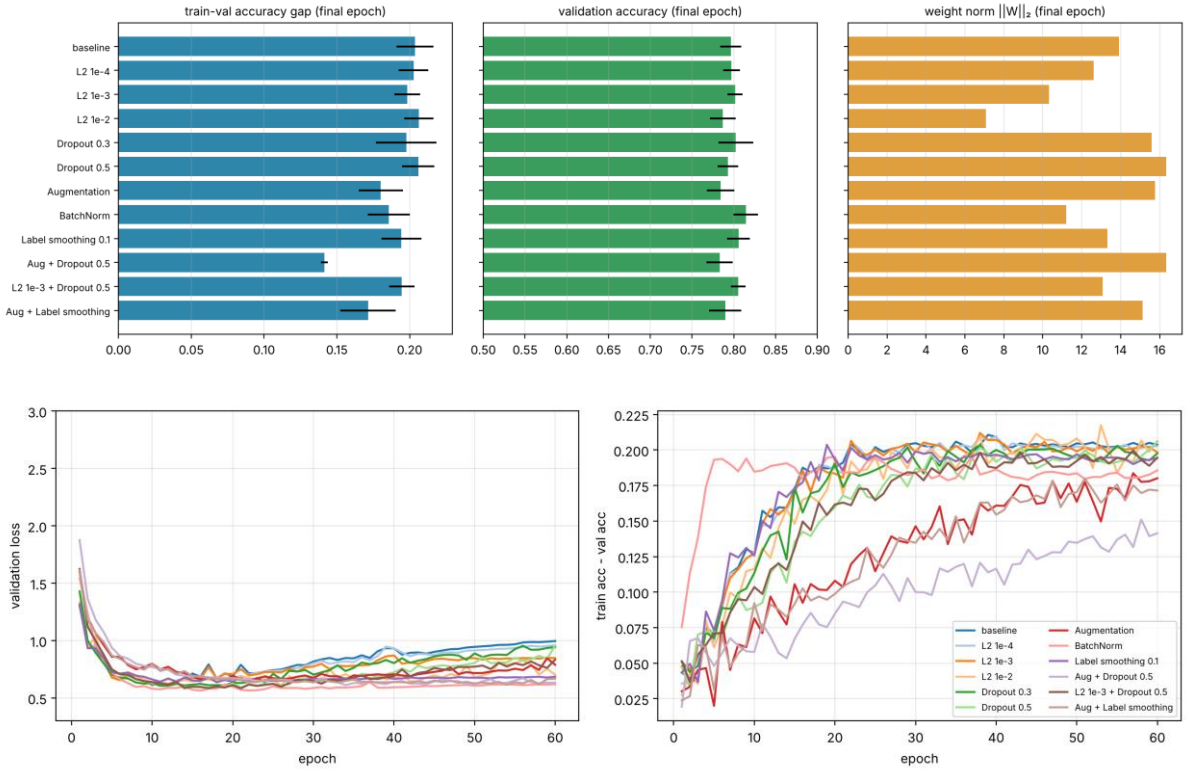
- A and C match the predictions (gap 0.042 ± 0.004 and 0.297 ± 0.007). In C validation loss is minimal at epoch 8.3 and then rises. Training accuracy is 0.998 ± 0.001 on the noisy labels but only 0.701 against the true labels, equal to validation accuracy.
- **B was wrong:** I predicted a gap of 25–35 points; observed 0.204 ± 0.013 (validation 0.796 ± 0.013). Fashion-MNIST classes are easier than I assumed. The mechanism is as predicted (train accuracy 100%, validation loss rising after epoch 12.7); only its size was wrong.
- Condition A was changed after a pilot: 10,000 images × 15 epochs gave a gap of 8–10 points, which is not “good” (log: logs/q3_pilot_first_attempt.log). I replaced it with 20,000 images × 6 epochs; the 3–6 point prediction was not rewritten before this second run.

3.3 Regularisers on condition B

L2 weight decay (Adam weight_decay 1e-4, 1e-3, 1e-2), dropout (0.3, 0.5, before the output layer), early stopping (weights of the epoch with minimum validation loss), data

augmentation (shift ±2 px and flip), batch normalisation, label smoothing (0.1), and three combinations. Weight norm = $\sqrt{\Sigma||W||^2}$ over weight matrices and kernels.

Regulariser	Train	Val. acc.	Gap (final)	Min val. loss	Epoch of min	Weight norm	Gap at min epoch	Test
baseline	**1.000**	**0.796 ± 0.013**	**0.204 ± 0.013**	**0.606**	**13**	**13.9**	**0.151**	**0.785**
L2 1e-4	1.000	0.797 ± 0.010	0.203 ± 0.010	0.602	11	12.6	0.136	0.786
L2 1e-3	1.000	0.802 ± 0.009	0.198 ± 0.009	0.604	12	10.3	0.151	0.783
L2 1e-2	0.993	0.787 ± 0.015	0.206 ± 0.010	0.585	18	7.1	0.165	0.765
Dropout 0.3	1.000	0.802 ± 0.021	0.198 ± 0.021	0.577	14	15.6	0.142	0.787
Dropout 0.5	0.999	0.793 ± 0.012	0.206 ± 0.011	0.574	15	16.3	0.117	0.779
Augmentation	0.964	0.784 ± 0.017	0.180 ± 0.015	0.627	31	15.8	0.122	0.767
BatchNorm	1.000	0.814 ± 0.015	0.186 ± 0.015	0.557	18	11.2	0.180	0.802
Label smoothing 0.1	1.000	0.806 ± 0.014	0.194 ± 0.014	0.617	17	13.3	0.166	0.794
Aug + Dropout 0.5	0.924	0.783 ± 0.016	0.141 ± 0.002	0.599	42	16.3	0.116	0.771
L2 1e-3 + Dropout 0.5	1.000	0.805 ± 0.009	0.195 ± 0.009	0.577	17	13.1	0.137	0.786
Aug + Label smoothing	0.961	0.790 ± 0.019	0.171 ± 0.019	0.599	51	15.1	0.164	0.770



The standard deviation over seeds is 1–2 points and the validation set has 1,000 images (standard error ≈ 1.3 points), so differences below about 2 points are not reliable and I do not claim them.

3.4 Inquiry

Why one regulariser reduced the gap more than another:

- **Augmentation** reduced the final gap most among single methods (0.180 vs 0.204). It lowers training accuracy (0.964) because the 300 images are never seen exactly twice.

Validation accuracy did not improve (0.784 vs 0.796): Fashion-MNIST test images are centred, so shift invariance adds little (a hypothesis I did not test).

- **Batch normalisation** gave the best validation accuracy (0.814 ± 0.015, test 0.802) and lowest validation loss (0.557) but no smaller gap (0.186). It helps optimisation, not memorisation. The gain is about one standard deviation.
- **L2:** λ = 1e-2 halved the weight norm (13.9 → 7.1) but the gap stayed 0.206 and validation accuracy fell. A smaller norm does not mean a smaller gap, because 300 points can still be fitted exactly.
- **Dropout** (only before the output layer) and **label smoothing** left the gap unchanged (0.198 – 0.206 and 0.194). The convolutional features can still memorise.
- **Early stopping:** at the epoch of minimum validation loss (mean epoch 13) the gap is 0.151 vs 0.204 ± 0.013 at epoch 60, but validation accuracy there (0.791) is not higher. The gap shrinks because training accuracy is lower. Validation loss rises (0.61 → 1.0) while accuracy stays flat, because the network becomes over-confident.
- Only information (augmentation) or stopping before memorisation (early stopping) visibly narrowed the gap. I predicted that augmentation would reduce it most and label smoothing would not change accuracy: both correct. I expected dropout and L2 to give moderate reductions: wrong.

Did combining two regularisers help?

- **Augmentation + dropout 0.5:** smallest gap (0.141 ± 0.002) but validation accuracy fell to 0.783 and test to 0.771 (baseline 0.785). The gap narrowed because training accuracy dropped to 0.924. It hurt accuracy slightly.
- **Augmentation + label smoothing:** gap 0.171, validation 0.790, test 0.770: no benefit beyond augmentation alone.
- **L2 1e-3 + dropout 0.5:** gap 0.195, validation 0.805: no difference from either alone.
- Both augmentation and dropout limit memorisation, so their effects overlap and the validation accuracy does not add up.

What the evidence says about judging a model by training accuracy:

- In B the baseline, L2, dropout, batch norm and label smoothing all have training accuracy ≈ 1.000 but validation accuracy from 0.787 to 0.814. Training accuracy cannot tell them apart.
- In C training accuracy reaches 0.998 ± 0.001 but the real accuracy on the same images (true labels) is only 0.701. A model that is “perfect” on its labels is the worse model (validation accuracy fell from about 0.83 at epoch 6 to 0.70).
- The method with the smallest gap (augmentation + dropout) did not have the best validation accuracy, so the gap alone is not enough either. Use a held-out set (larger than 1,000 images), watch validation loss, and stop early (about epoch 10–20 here).

Q4 – Optimisation for Training Deep Models: Which Optimiser, Under Which Conditions?

4.1 Problem and data

Nightly retraining of a demand forecaster within a fixed budget. A public load series was not reachable, so a synthetic hourly series (3 years, 26,280 hours) was generated: daily double peaks, weekday and holiday effects, annual season, temperature-driven demand, AR(1) fluctuation and 10% measurement noise. 13 features (hour, weekday, month as sine/cosine, weekend and holiday flags, temperature, humidity, load 1 h, 23 h and 167 h earlier) predict load(t+1). Chronological split 17,353 / 4,368 / 4,391. Network $13 \to 64 \to 64 \to 64 \to 1$ (ReLU, He, 9,281 parameters). Loss is MSE in standardised units, so validation MSE = $1 - R^2$. Noise floor 0.0097; linear regression reaches 0.0565.

4.2 Update rules and prediction

Optimiser	Update rule (g = mini-batch gradient)
SGD	$\theta \leftarrow \theta - \eta\, g$
Momentum ($\mu = 0.9$)	$v \leftarrow \mu v + g;\ \theta \leftarrow \theta - \eta\, v$
RMSProp ($\rho = 0.9$)	$s \leftarrow \rho s + (1-\rho) g^2;\ \theta \leftarrow \theta - \eta\, g / (\sqrt{s} + \varepsilon)$
Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)	$m \leftarrow \beta_1 m + (1-\beta_1) g;\ v \leftarrow \beta_2 v + (1-\beta_2) g^2;\ \theta \leftarrow \theta - \eta\, \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$, with bias-corrected $\hat{m}$, $\hat{v}$

Prediction: Adam fastest, then RMSProp, momentum, SGD. At lr 0.1 SGD and momentum diverge, RMSProp and Adam oscillate (their step is about η regardless of gradient size). Most of Adam's advantage should come from per-parameter adaptation.

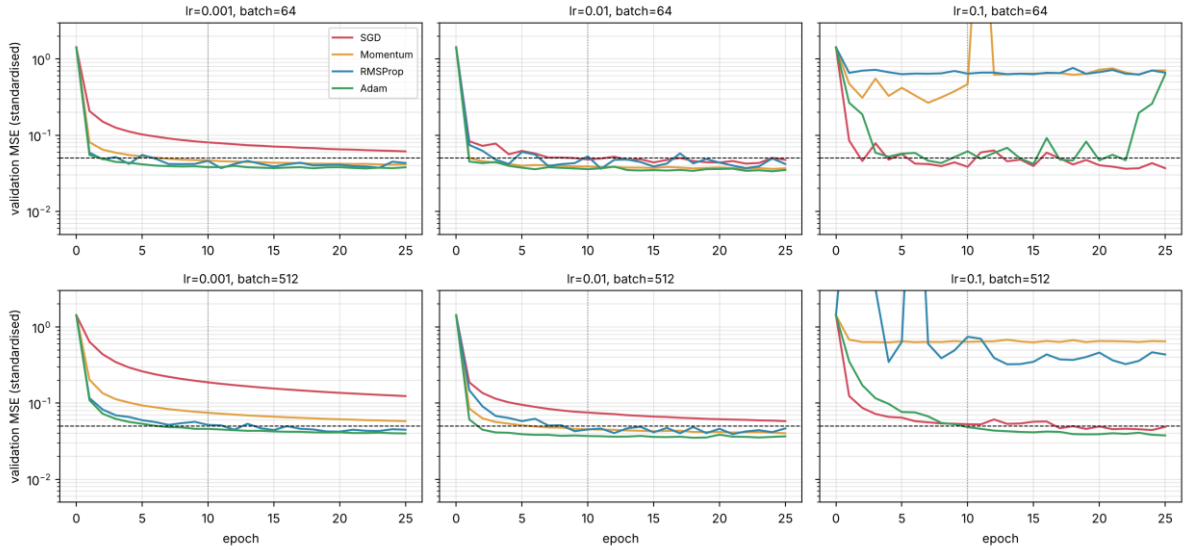
4.3 Experiment

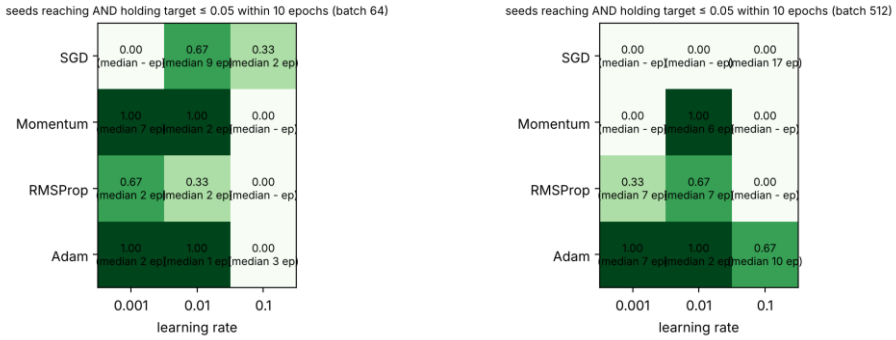
4 optimisers × lr {1e-3, 1e-2, 1e-1} × batch {64, 512} × 3 seeds = 72 runs of 25 epochs. Target: validation MSE ≤ 0.05 ($R^2 \geq 0.95$; the pilot showed that the first choice of 0.03 was too strict for the budget). Nightly budget: 10 epochs. A run is a reliable success if it reaches the target within the budget and still satisfies it at epoch 10. Spikes = epochs in which validation MSE rises by more than 10%.

Optimiser	lr	Batch	Reliable	Epochs to target	Final RMSE	Final MAE	Spikes	Diverged	Sec to target
SGD	0.001	64	0.00	–	0.290	0.228	0.0	0/3	–
SGD	0.001	512	0.00	–	0.412	0.325	0.0	0/3	–
SGD	0.01	64	0.67	9	0.255	0.200	3.7	0/3	1.19
SGD	0.01	512	0.00	–	0.282	0.222	0.0	0/3	–
SGD	0.1	64	0.33	2	0.225	0.176	2.3	2/3	0.26
SGD	0.1	512	0.00	17	0.259	0.203	0.7	2/3	0.50
Momentum	**0.001**	**64**	**1.00**	**7**	**0.238**	**0.187**	**0.0**	**0/3**	**1.15**
Momentum	0.001	512	0.00	–	0.282	0.222	0.0	0/3	–
Momentum	**0.01**	**64**	**1.00**	**2**	**0.225**	**0.175**	**0.0**	**0/3**	**0.33**
Momentum	**0.01**	**512**	**1.00**	**6**	**0.235**	**0.184**	**0.0**	**0/3**	**0.21**
Momentum	0.1	64	0.00	–	diverged	diverged	2.7	2/3	–
Momentum	0.1	512	0.00	–	diverged	diverged	0.0	2/3	–
RMSProp	0.001	64	0.67	2	0.243	0.189	4.0	0/3	0.43

Optimiser	lr	Batch	Reliable	Epochs to target	Final RMSE	Final MAE	Spikes	Diverged	Sec to target
RMSProp	0.001	512	0.33	7	0.247	0.194	5.7	0/3	0.22
RMSProp	0.01	64	0.33	2	0.239	0.185	9.3	0/3	0.47
RMSProp	0.01	512	0.67	7	0.252	0.192	6.3	0/3	0.29
RMSProp	0.1	64	0.00	–	0.952	0.773	4.7	0/3	–
RMSProp	0.1	512	0.00	–	0.761	0.617	7.3	0/3	–
Adam	**0.001**	**64**	**1.00**	**2**	**0.228**	**0.178**	**0.7**	**0/3**	**0.45**
Adam	**0.001**	**512**	**1.00**	**7**	**0.234**	**0.183**	**0.0**	**0/3**	**0.29**
Adam	**0.01**	**64**	**1.00**	**1**	**0.219**	**0.170**	**2.0**	**0/3**	**0.24**
Adam	**0.01**	**512**	**1.00**	**2**	**0.224**	**0.174**	**0.3**	**0/3**	**0.07**
Adam	0.1	64	0.00	3	0.929	0.757	9.3	0/3	0.87
Adam	0.1	512	0.67	10	0.227	0.177	1.0	0/3	0.64

Bold rows: reliable in 3/3 seeds. “Reliable” is the fraction of seeds that reach and hold the target.





4.4 Convergence inquiry

- Settings (out of 6 per optimiser) that succeed in all 3 seeds: **Adam 4, Momentum 3, SGD 0, RMSProp 0.**
- Adam works at lr 1e-3 and 1e-2 for both batch sizes. At 1e-1 it fails with batch 64 and is unreliable (2/3 seeds) with batch 512.
- Momentum works at 1e-2 (both batches) and 1e-3 with batch 64. SGD at lr 0.01 reaches the target in 2 of 3 seeds (median 9 epochs); at 0.1 it diverges in 2 of 3 seeds.

- RMSProp touches the target but oscillates (4–9 spikes per run), so it does not hold it at epoch 10 in every seed.

Optimiser	Step length $\|\|\Delta\theta\|\|$	$\cos(\Delta\theta, -g)$	CV($\|\Delta\theta\|$) / CV($\|g\|$)
SGD	0.0174	1.000	1.00
Momentum	0.0253	0.412	0.83
RMSProp	0.5306	0.420	0.57
Adam	0.0811	0.188	0.61

Decomposition of an update at lr 0.01, batch 64 (first 200 updates, seed 0).

- **Step size** decides stability: SGD at lr 0.1 diverges; RMSProp's step is about η per parameter ($||\Delta\theta|| = 0.53$, 30× SGD's), which explains its jitter.
- **Direction:** momentum at lr 0.01 has the same effective step as SGD at lr 0.1 ($\eta/(1-\mu)$), yet converged in 2 epochs in 3/3 seeds while SGD at 0.1 diverged. Averaging past gradients (cos = 0.41) cancels mini-batch noise, so the gain of momentum comes from the direction.
- **Per-parameter adaptation:** RMSProp and Adam compress the spread of step sizes (0.57 and 0.61 vs 1.00 for SGD), so parameters with small gradients still move. Adam combines this with momentum and tolerates two learning rates.

Recommendation

Adam, lr 1e-2, batch 512: reaches $R^2 \geq 0.95$ in a median of 2 epochs (0.07 s) in 3/3 seeds, final RMSE 0.224 (standardised units). Batch 64 needs 1 epoch but about 3× more wall-clock time (0.24 s). Lr 1e-3 is also reliable. Fallback: momentum, lr 1e-2, batch 64 (2 epochs, 3/3 seeds). Never lr 1e-1.

Predicted vs observed: correct for Adam fastest and most robust, SGD and momentum diverging at lr 0.1, RMSProp and Adam not settling at lr 0.1 with batch 64 (RMSE 0.95 and 0.93). Wrong: RMSProp was not second; the tuned momentum optimiser is nearly as good as Adam (2 epochs vs 1). Most of the speed-up over SGD is due to direction smoothing; the extra gain of Adam is a wider range of usable learning rates.

Q5 – Derive a Convolutional Network from a Design Principle

5.1 Setup

PCB images are not reachable, so Fashion-MNIST (28×28, 10 classes) is used. Split 20,000 / 5,000 / 10,000 (official test set). All models are trained on un-shifted images only: Adam, lr 1e-3, batch 128, 10 epochs, 3 seeds.

5.2 From the principle to an architecture

- **Local detector:** a k×k kernel K with bias b slides over the image: $y[i, j] = b + \Sigma_m \Sigma_n K[m, n] \cdot x[i \cdot s + m - p, j \cdot s + n - p]$ (also summed over input channels). C_out kernels give C_out feature maps.
- **Output size:** $n_{out} = \lfloor (n + 2p - k) / s \rfloor + 1$. For n = 28: k = 3, p = 1, s = 1 → 28; k = 3, p = 0, s = 1 → 26; k = 3, p = 1, s = 2 → 14; pool 2×2, s = 2 → 14.
- **Weight sharing:** one kernel ($k^2 \cdot C_{in} + 1$ parameters) is reused at all positions, so the parameter count does not depend on the image size.

- **Pooling:** max-pool 2×2 keeps the strongest response in a block. A one-pixel shift often leaves the block maximum unchanged, so small shifts are tolerated (shifts larger than the window are not).
- **Classification:** flatten (or global average pool) and a linear layer with softmax.

Model	Layers	Params	MACs
FC	Flatten → Linear(784, 256) → ReLU → Linear(256, 128) → ReLU → Linear(128, 10)	235,146	234,752
CNN	Conv(1→16, 3×3, p 1) → ReLU → MaxPool 2 → Conv(16→32, 3×3, p 1) → ReLU → MaxPool 2 → Flatten (1,568) → Linear(1568, 10)	20,490	1,031,744
CNN-GAP	as CNN, then Conv(32→64, 3×3, p 1) → ReLU → global average pool → Linear(64, 10)	23,946	1,919,872

5.3 Parameter derivation for the same 28 × 28 input

Fully connected first layer (256 units on 784 pixels): P_FC = 784 × 256 + 256 = 200,960
Convolutional first layer (16 kernels of 3×3×1): P_conv = (3 × 3 × 1 + 1) × 16 = 160
Ratio = 200,960 / 160 = 1,256×. For 224×224×3 inputs the same FC layer would need 38.5 M weights; P_conv would not change.
CNN total = 160 + (3·3·16 + 1)·32 + (1568·10 + 10) = 160 + 4,640 + 15,690 = 20,490
CNN-GAP total = 160 + 4,640 + (3·3·32 + 1)·64 + (64·10 + 10) = 160 + 4,640 + 18,496 + 650 = 23,946

The counts agree with PyTorch.

5.4 Parameters, accuracy and training time

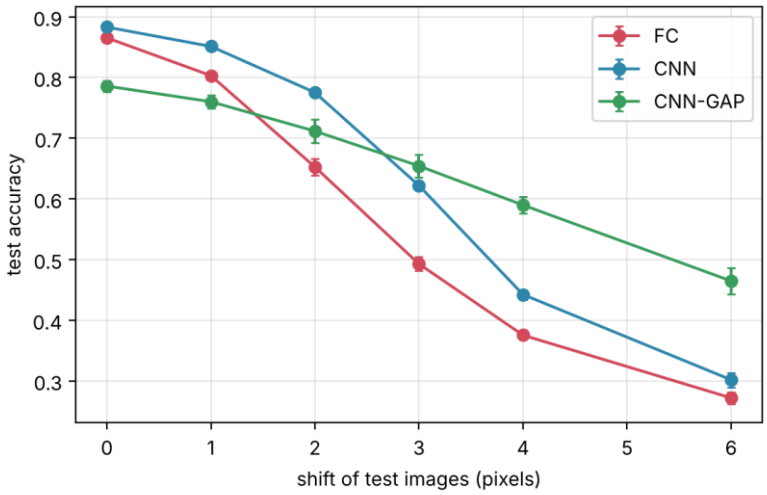
Model	Parameters	MACs / image	Train time (s)	Test accuracy
FC	235,146	234,752	3.7	0.865 ± 0.003
CNN	20,490	1,031,744	40.2	0.883 ± 0.002
CNN-GAP	23,946	1,919,872	55.1	0.786 ± 0.009

- The CNN has 11.5× fewer parameters and is about 1.8 points more accurate (0.883 ± 0.002 vs 0.865 ± 0.003), but trains about 11× slower (40.2 s vs 3.7 s, CPU), because it uses about 4.4× more multiply–accumulates. Weight sharing saves parameters, not computation.
- CNN-GAP is the least accurate on centred images (0.786 ± 0.009). Its head averages over all positions and discards the layout (my reading, not tested separately).

5.5 Translation test

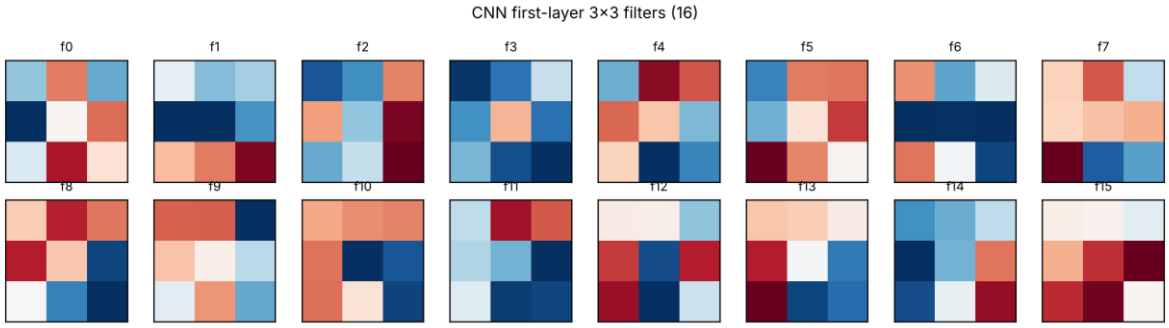
Test images are shifted by k pixels in 4 directions (empty pixels filled with background; content shifted out is lost); accuracy is averaged over the directions.

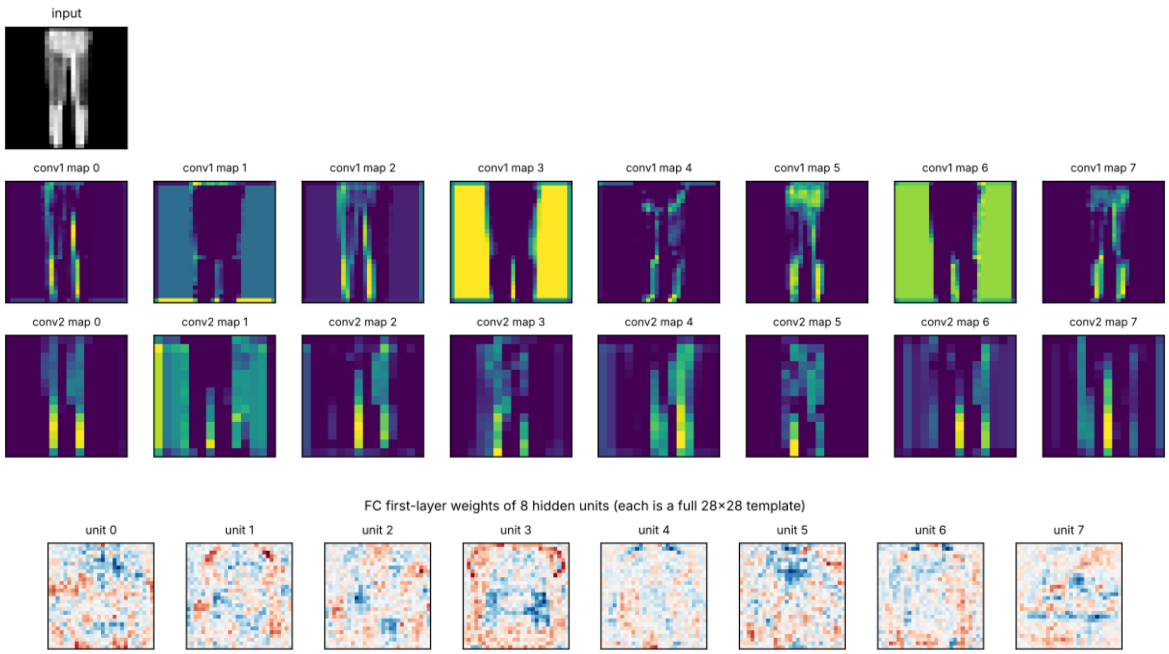
Model	0 px	1 px	2 px	3 px	4 px	6 px
FC	0.865	0.803	0.652	0.493	0.375	0.272
CNN	0.883	0.851	0.775	0.622	0.442	0.302
CNN-GAP	0.786	0.760	0.712	0.654	0.590	0.465



- Drop in accuracy (points) at 2 px: FC 21.3, CNN 10.8, CNN-GAP 7.4; at 4 px: 49.0, 44.1, 19.6; at 6 px: 59.3, 58.1, 32.1.
- For small shifts (1–3 px) the CNN loses clearly less than the FC network (about half at 2 px), as the principle predicts.
- The advantage is limited: the CNN with a flatten head is at 0.44 at 4 px and loses as much as the FC network at 6 px (58 vs 59 points). The flatten layer is fully connected over positions.
- CNN-GAP is the most shift-tolerant (drop 32 points at 6 px) although it is the least accurate on centred images: a trade-off between positional information and tolerance.
- Fashion-MNIST objects fill most of the frame, so large shifts also push parts of the object out of view. Part of the loss at 4–6 px is lost information, not model failure.

5.6 Learned filters and feature maps





Several first-layer filters look like oriented edge detectors (f1 and f6 have a row of strongly negative weights, f14 a column). In the feature maps of the sample (a trouser), conv1 maps 0, 2, 4 and 7 respond along the vertical leg edges and map 5 to the waist; maps 1, 3 and 6 are nearly constant on the background (driven by the bias). Conv2 maps are coarser (7×7). The FC weights (last figure) are noisy templates tied to absolute positions. The description is qualitative; I did not measure how many filters are edge-like.

5.7 Correctness inquiry: why does weight sharing detect a defect at an unseen position?

Convolution: $y[i, j] = \Sigma\, K[m, n] \cdot x[i + m, j + n]$
Shifted image: $x'[i, j] = x[i - a, j - b]$
$y'[i, j] = \Sigma\, K[m, n] \cdot x[i - a + m, j - b + n] = y[i - a, j - b]$

Shifting the input shifts the feature map by the same amount and changes nothing else (translation equivariance). The same K is used at every position, so a kernel that learned a scratch at the top-left gives exactly the same response at the bottom-right: the response depends on the patch content, not its location. A fully connected unit has a separate weight for every pixel; weights at positions where no defect appeared in training get no useful signal. That is why the conv layer needs 160 parameters instead of 200,960.

Numerical check: for the first conv of the trained CNN (no bias, interior pixels, 3-pixel shift) $\max |\text{conv}(\text{shift } x) - \text{shift}(\text{conv } x)| = 0.00\text{e+}0$. Equivariance of the feature maps is not invariance of the prediction: max-pool adds only local tolerance and the flatten + linear head is position-specific, which is where the CNN lost accuracy at 4–6 px. Global average pooling removes the position dependence, so CNN-GAP degraded least.

Q6 – Architecture Design: Capacity, Cost and the Deployment Boundary

6.1 Problem and architecture family

On-device wildlife camera trap. Fashion-MNIST resized to 16×16 or 32×32 (bilinear) stands in for animal images: 10,000 / 5,000 / 10,000. My family Net(d, w, r) has depth d (3×3 conv layers), width w (filters) and input resolution r:

Layer	Operation	Output
1	conv 3×3 (1 → w, pad 1) + ReLU, max-pool 2×2	w × r/2 × r/2
2	conv 3×3 (w → w) + ReLU, max-pool 2×2	w × r/4 × r/4
3 … d	(d − 2) × conv 3×3 (w → w) + ReLU	w × r/4 × r/4
head	global average pool, Linear (w → 10)	10

Design iteration: a first family with one pooling stage and r ∈ {14, 28} gave clearly lower accuracy because the receptive field of a two-layer network was too small. I replaced it by pooling after each of the first two layers and r ∈ {16, 32}.

6.2 Formulas

Parameters = $(9w + w) + (d-1)(9w^2 + w) + (10w + 10)$
MACs = $9 \cdot w \cdot r^2 + 9 \cdot w^2 \cdot (r/2)^2 + (d-2) \cdot 9 \cdot w^2 \cdot (r/4)^2 + 10 \cdot w$ (bias additions not counted)
Receptive field: RF_l = RF_{l−1} + (k_l − 1)·j_{l−1}, j_l = j_{l−1}·s_l → RF = 8d − 6 (10 px for d = 2, 26 px for d = 4)

6.3 Correctness check: hand calculation vs framework (d = 2, w = 8, r = 16)

Layer	Output shape	Parameters	MACs
input	1×16×16	0	0
conv1 3x3	8×16×16	80	18,432
maxpool 2x2	8×8×8	0	0
conv2 3x3	8×8×8	584	36,864
maxpool 2x2	8×4×4	0	0
global avg pool	8	0	0
linear	10	90	80

Hand total: 754 parameters and 55,376 MACs. PyTorch reports 754 parameters; an independent forward-hook counter reports 55,376 MACs: exact match. torchinfo prints 57,946 because it also counts the bias additions (8·16·16 + 8·8·8 + 10 = 2,570). The formulas were asserted against the hook counter for all 7 architectures.

6.4 Growth prediction (before the experiments)

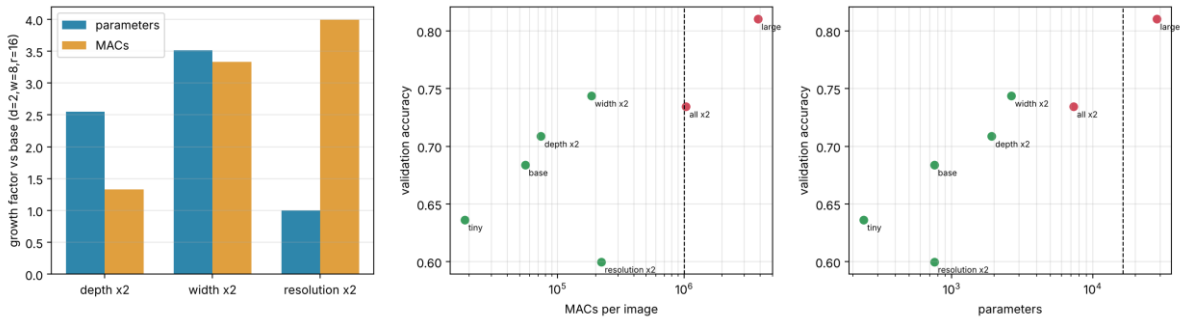
Width multiplies both C_in and C_out, so doubling it multiplies interior parameters and MACs by 4. Resolution multiplies every conv layer by 4 (H·W) and leaves parameters unchanged (global pooling). Depth repeats layers, so it gives about ×2 parameters and less than ×2 MACs, because the added layers run on the smallest map. Prediction: resolution and width are the expensive dimensions, depth is cheap.

Doubling …	Parameters	MACs
depth x2	2.55×	1.33×
width x2	3.51×	3.33×
resolution x2	1.00×	4.00×
depth x2 (from d=8,w=32,r=32)	2.13×	1.76×
width x2 (from d=8,w=32,r=32)	3.97×	3.90×
resolution x2 (from d=8,w=32,r=32)	1.00×	4.00×

6.5 Experiment

Seven architectures, Adam lr 3e-3, batch 128, 10 epochs, 3 seeds. MACs by hooks, file size = float32 state_dict, latency = median of 5 × 500 batch-1 forward passes on one CPU thread (idle machine, untrained weights).

Model	d/w/r	Params	MACs	KB	RF	ms	Val. accuracy	Test	Fits
tiny	2/4/16	238	18,472	3.7	10	0.058	0.636 ± 0.014	0.650	yes
base	2/8/16	754	55,376	5.6	10	0.059	0.684 ± 0.002	0.693	yes
depth x2	4/8/16	1,922	73,808	11.1	26	0.088	0.709 ± 0.019	0.714	yes
width x2	2/16/16	2,650	184,480	13.0	10	0.069	0.744 ± 0.005	0.749	yes
resolution x2	2/8/32	754	221,264	5.6	10	0.086	0.600 ± 0.043	0.612	yes
all x2	4/16/32	7,290	1,032,352	31.9	26	0.227	0.734 ± 0.013	0.743	no
large	4/32/32	28,394	3,834,176	114.3	26	0.441	0.810 ± 0.014	0.812	no



- Prediction confirmed: from the base, resolution ×2 gives ×4.00 MACs and ×1.00 parameters; width ×2 gives ×3.33 MACs and ×3.51 parameters (below 4 because the first layer and head grow only linearly; asymptotically 3.9 – 4.0); depth ×2 gives ×1.33 MACs and ×2.55 parameters. My earlier guess “depth ×2 → MACs ×2” was too high.
- Accuracy did not follow cost: doubling the resolution lowered validation accuracy (0.684 → 0.600), because the receptive field stays 10 px while the object grows and the 32×32 images are only up-sampled 28×28 images. Real camera-trap images could behave differently.

6.6 Why does doubling the width cost more than doubling the depth?

MACs of one layer = $k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$. Width multiplies both C_in and C_out (×4 per layer for the same number of layers). Depth only repeats layers (×2 at most), and here the added layers run on the smallest feature map (×1.33 MACs measured).

6.7 Deployment decision

Assumed device (fixed before the experiments): microcontroller class, about 50 M MAC/s, 256 KB flash. Budget: ≤ 64 KB of float32 weights (a quarter of the flash) and ≤ 1.0 M MACs (about 20 ms per frame). Five of the seven models fit; "all ×2" (1.03 M MACs) and "large" (3.8 M MACs, 114 KB) do not.

- **Deploy "width x2" (d = 2, w = 16, r = 16):** the most accurate model inside the budget. Validation 0.744, test 0.749, 2,650 parameters (13 KB), 0.18 M MACs.
- **Sacrificed:** 6.7 accuracy points on validation (6.2 on test) compared with the largest model ("large", validation 0.810, test 0.812).
- "all ×2" exceeds the MAC budget by only 3% but is also less accurate (0.734): extra computation spent on resolution is a poor investment here. "depth ×2" (11 KB, 0.71) is the next candidate.
- Caveat: latency was measured on this workstation, not on the target microcontroller; MACs and bytes transfer, milliseconds do not. Real forest images (light, occlusion, empty frames) will lower the accuracy of every model.

Q7 – Diagnose, Redesign and Validate

Model selected from Q1–Q6: the 12-hidden-layer sigmoid network (width 64, Xavier init) for fraud screening from Q1. It has the clearest limitation, vanishing gradients. The redesign uses the same data, seeds (0–2), optimiser and 15-epoch budget as Q1.

A. Diagnosis

- **Where:** in the layers close to the input, from the first update. Layer-wise gradient norms rise geometrically from 4.9e-8 (layer 1) to 0.37 (output). The loss stays at 0.473 → 0.466 and validation recall is 0 in every seed: it is present in all epochs.
- **Which quantity shrinks:** the gradient. $\|g_1\|/\|g_out\|$ = 1.34e-7, a loss of 0.252 per layer (mean σ' at init = 0.235).
- **Root cause: architecture, not capacity, data or optimiser.** The same width and data train well with ReLU and tanh (Q1). No learning rate can fix a layer-1 gradient that is 1e-7 of the output gradient. In Q1 the sigmoid net with N(0,1) weights (gain near 1) did train.
- **Responsible characteristic:** $\sigma' \leq 1/4$ with unit-gain initialisation, so each layer multiplies the gradient by about 1/4 along a single path with no identity shortcut: $(1/4)^{11} \approx$ 2.4e-7.

B. Design objectives (measurable, same 15-epoch budget)

#	Objective	Original
1	$\|g_1\|/\|g_out\| \geq$ 1e-2 (first 50 updates)	1.34e-7
2	Validation recall ≥ 0.80 and AUPRC ≥ 0.85	recall 0.00, AUPRC 0.07
3	No additional parameters	47,809
4	Training time (per step) ≤ 1.5 × original	1.73 ms

C. Design of the revised model

Principle: keep the per-layer Jacobian gain close to 1, either with a variance-preserving unit (ReLU with He: $E[\varphi'^2] \cdot n \cdot \mathrm{Var}(W) = \tfrac{1}{2} \cdot n \cdot (2/n) = 1$) or with an identity path. Four variants separate the causes:

Design	Activation / init	Skip	Tests
Original	sigmoid, Xavier	no	baseline
R1	sigmoid, Xavier × 4	no	gain correction only
R2	ReLU, He	no	variance-preserving unit
R3	sigmoid, Xavier	yes	identity gradient path
R4 (final)	**ReLU, He**	**yes**	**both principles**

R4: Linear(30, 64) → ReLU → h_1; for $l = 2 \ldots 12$: $h_l = h_{l-1} + \alpha \cdot \mathrm{ReLU}(W_l h_{l-1} + b_l)$ with $\alpha = 1/\sqrt{11} \approx 0.30$; output Linear(64, 1). Biases 0, weights He-normal. Parameters: 1,984 + 11 × 4,160 + 65 = 47,809, identical to the original.

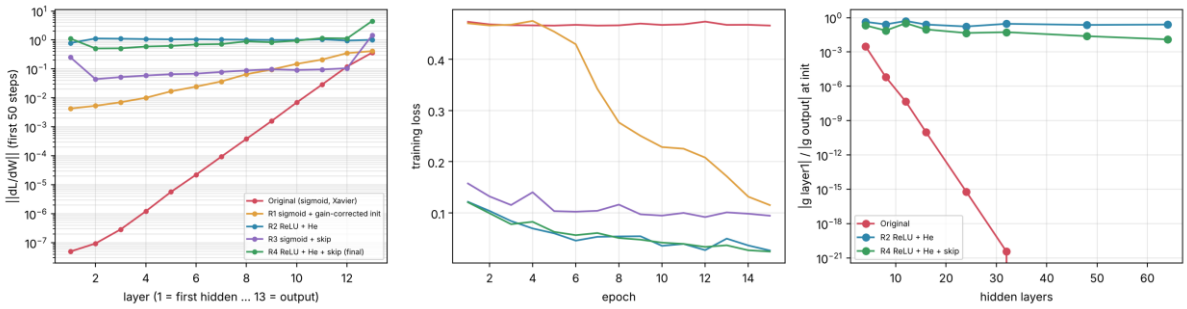
Exploited: the unit and initialisation keep the forward and backward variance constant, and the skip gives the gradient a path of unit gain. Avoided: the product of ¼-factors. Paid: one vector addition per layer, no extra parameters. New trade-offs: ReLU units can die, outputs are unbounded, and residual sums grow (hence α).

D. Prediction before implementation

Recorded in results/q7_prediction_before_running.md: R4 ratio 0.3 – 3, training loss ≤ 0.03, recall ≥ 0.8, dead ReLU < 5%, same parameters, < 10% time overhead. R1 trains but is brittle (ratio 1e-2 – 1, recall ≈ 0.8). R3 ratio 0.1 – 1 but slightly worse than R4. The skip is not needed at 12 layers (R2 ≈ R4); its benefit would appear at much larger depth.

E. Implementation and validation

Design	Params	$\|\|g_1\|\|/\|\|g_{out}\|\|$	Train loss	Val. loss	Val. recall	Val. AUPRC	Dead ReLU	ms/step
Original (sigmoid, Xavier)	47,809	1.29e-07	0.4661	0.465	0.000 ± 0.000	0.068 ± 0.055	n/a	1.73
R1 sigmoid + gain-corrected init	47,809	9.95e-03	0.1153	0.118	0.827 ± 0.042	0.859 ± 0.016	n/a	2.46
R2 ReLU + He	47,809	7.96e-01	0.0265	0.173	0.881 ± 0.035	0.881 ± 0.015	2.8%	2.31
R3 sigmoid + skip	47,809	1.77e-01	0.0945	0.081	0.895 ± 0.013	0.915 ± 0.001	n/a	2.26
R4 ReLU + He + skip (final)	**47,809**	**2.47e-01**	**0.0244**	**0.112**	**0.867 ± 0.017**	**0.893 ± 0.014**	**2.5%**	**2.28**



Depth stress test at initialisation ($||g_1||/||g_out||$; only the Original, R2 and R4 designs were included):

Hidden layers	Original (sigmoid, Xavier)	R2 ReLU + He	R4 ReLU + He + skip (final)
4	3.05e-03	4.25e-01	2.09e-01
8	6.48e-06	2.59e-01	7.43e-02
12	4.48e-08	5.14e-01	3.26e-01
16	9.83e-11	2.50e-01	9.37e-02
24	6.15e-16	1.69e-01	4.55e-02
32	3.54e-21	2.89e-01	5.40e-02
48	0 (underflow)	2.31e-01	2.41e-02
64	0 (underflow)	2.49e-01	1.25e-02

F. Defence of the final design

- **Objective 1:** original 1.29e-7 → R4 2.47e-1. R2 7.96e-1, R3 1.77e-1, R1 9.95e-3 (just below 1e-2). Met by R2, R3, R4; missed narrowly by R1.
- **Objective 2:** original recall 0.00, AUPRC 0.07 → R4 recall 0.87 ± 0.02, AUPRC 0.89 ± 0.01, training loss 0.466 → 0.024. Met (R1 0.83 / 0.86, R2 0.88 / 0.88, R3 0.89 / 0.91). Test recall of R4: 0.90 (precision 0.77).
- **Objectives 3 and 4:** parameters identical (47,809). 2.28 ms/step vs 1.73 ms (+31%), inside ×1.5. R1 is +42% with no extra operations, so these shared-CPU timings are mostly noise.
- **Prediction check:** supported: R4 training loss 0.024 ≤ 0.03, recall ≥ 0.8, dead units 2.5%, R3 ratio 0.18, R1 recall 0.83, R2 ≈ R4. Not supported: **R3 trained better than R4**; R4's ratio (0.25) is slightly below 0.3 – 3; R1's ratio is at the edge of 1e-2 – 1; and the stress test shows **no advantage of the skip**: ReLU + He without skips stays roughly flat (0.17 – 0.51 from 4 to 64 layers) while R4 decays from 0.21 to 0.0125 (not investigated; a hypothesis is that the α-scaled branches shrink the identity path's share).
- **Significance:** the improvement over the original is huge (recall 0 → 0.87). Among working designs R3 has the best AUPRC (0.915 ± 0.001) and lowest validation loss (0.081); R4 (0.893 ± 0.014) and R2 (0.881 ± 0.015) are within about two standard deviations. With 3 seeds and 98 validation frauds I do not claim R4 is better than R2 or R3. R4 was fixed as the final design before the runs; the data do not contradict it but do not single it out.
- **When the original may still be preferable:** shallow networks (3 layers, Q1), bounded hidden outputs (gating units), or hardware lookup tables. For depth ≥ 8 it is not usable.
- **Conclusion:** the redesign addresses the limitation. Correcting the gain alone (R1), the unit alone (R2) or the skip alone (R3) each restores training (recall 0 → 0.83 – 0.90). The simplest sufficient fix is R2 (ReLU + He). The extra benefit of the skip at 12 layers was not demonstrated, and at 64 layers it was worse. Limits: 3 seeds, one dataset, threshold 0.5.

Appendix A – Code

Complete scripts (also in the zip: code/). Outputs are in the logs/ folder and results/*.csv|json of the package; every table above was generated from those files by build_report_data.py.

common.py – data loaders, seeds, logger

```
"""Shared utilities: seeding, data loaders, logging.  CSA47 Deep Learning - Assignment 2
(CO2)."""
import os, gzip, json, random, time
import numpy as np
import pandas as pd
import torch

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
DATA = os.path.join(ROOT, "data")
RES = os.path.join(ROOT, "results")
FIG = os.path.join(ROOT, "figures")
LOG = os.path.join(ROOT, "logs")
for d in (RES, FIG, LOG):
    os.makedirs(d, exist_ok=True)

SEEDS = [0, 1, 2]          # every stochastic experiment is repeated for these seeds (mean
reported)
torch.set_num_threads(1)   # deterministic-ish and lets two scripts share the 2 CPU cores


def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)



class Logger:
    """Tee to console and to logs/<name>.log"""
    def __init__(self, name):
        self.f = open(os.path.join(LOG, name + ".log"), "w")
        self.t0 = time.time()

    def __call__(self, *a):
        s = " ".join(str(x) for x in a)
        line = f"[{time.time()-self.t0:7.1f}s] {s}"
        print(line, flush=True)
        self.f.write(line + "\n")
        self.f.flush()



# ------------------------------------------------------------------------- Fashion-MNIST
def _idx(path, images):
    with gzip.open(path, "rb") as f:
        buf = f.read()
    if images:
        return np.frombuffer(buf, np.uint8, offset=16).reshape(-1, 28, 28)
    return np.frombuffer(buf, np.uint8, offset=8)



def load_fmnist():
    """Returns uint8 arrays: Xtr (60000,28,28), ytr, Xte (10000,28,28), yte."""
    return (_idx(f"{DATA}/train-images-idx3-ubyte.gz", True), _idx(f"{DATA}/train-labels-idx1-
ubyte.gz", False),
            _idx(f"{DATA}/t10k-images-idx3-ubyte.gz", True), _idx(f"{DATA}/t10k-labels-idx1-
ubyte.gz", False))



FM_MEAN, FM_STD = 0.2860, 0.3530
```

```
def fmnist_split(n_train, n_val, seed=0, balanced=True):
    """Fixed split: n_train from the 60k training pool, n_val DISJOINT validation images from
the same
    pool, official 10k test set. Returns float tensors (N,1,28,28) normalised."""
    Xtr, ytr, Xte, yte = load_fmnist()
    rng = np.random.RandomState(1234 + seed)
    perm = rng.permutation(len(Xtr))
    val_idx = perm[:n_val]
    tr_idx = perm[n_val:n_val + n_train]

    def prep(X):
        return ((torch.from_numpy(X.astype(np.float32)) / 255.0 - FM_MEAN) /
FM_STD).unsqueeze(1)
    return (prep(Xtr[tr_idx]), torch.from_numpy(ytr[tr_idx].astype(np.int64)),
            prep(Xtr[val_idx]), torch.from_numpy(ytr[val_idx].astype(np.int64)),
            prep(Xte), torch.from_numpy(yte.astype(np.int64)))



# ---------------------------------------------------------------------------- credit-card
fraud
def load_fraud(n_normal=30000, seed=0):
    """Credit-card fraud (ULB / Kaggle, 284,807 transactions, 492 frauds).
    Sub-sample for CPU budget: all 492 frauds + `n_normal` random legitimate transactions
    (fraud rate stays rare, 1.6%).  Stratified 60/20/20 split.  Features standardised on the
training part."""
    df = pd.read_csv(f"{DATA}/creditcard.csv")
    rng = np.random.RandomState(777)  # fixed sub-sample / split for ALL runs (seed varies init
& batching only)
    fr = df[df.Class == 1]
    nm = df[df.Class == 0].iloc[rng.permutation((df.Class == 0).sum())[:n_normal]]
    d = pd.concat([fr, nm]).reset_index(drop=True)
    y = d.Class.values.astype(np.float32)
    X = d.drop(columns=["Class"]).values.astype(np.float32)
    idx = rng.permutation(len(d))
    splits = {}
    # stratified: split each class separately
    parts = {"tr": [], "va": [], "te": []}
    for c in (0, 1):
        ci = idx[y[idx] == c]
        a, b = int(0.6 * len(ci)), int(0.8 * len(ci))
        parts["tr"].append(ci[:a]); parts["va"].append(ci[a:b]); parts["te"].append(ci[b:])
    out = {}
    for k in parts:
        out[k] = np.concatenate(parts[k]); rng.shuffle(out[k])
    mu, sd = X[out["tr"]].mean(0), X[out["tr"]].std(0) + 1e-8
    T = lambda k: (torch.from_numpy((X[out[k]] - mu) / sd), torch.from_numpy(y[out[k]]))
    return T("tr"), T("va"), T("te")



def batches(X, y, bs, gen):
    n = len(X)
    perm = torch.randperm(n, generator=gen)
    for i in range(0, n, bs):
        j = perm[i:i + bs]
        yield X[j], y[j]



def save_json(obj, name):
    with open(os.path.join(RES, name), "w") as f:
        json.dump(obj, f, indent=2)
```

q1_activations.py – Q1 (and imported by Q7)

```
"""Q1 - Hidden units: does the activation function matter as the network grows deeper?
Credit-card fraud screening (ULB/Kaggle dataset, sub-sampled; see common.load_fraud).
```

```
Configurations (width 64 everywhere, 30 inputs, 1 logit output):
  shallow   : 3 hidden layers,  proper init (Xavier for sigmoid/tanh, He for ReLU)
  deep      : 12 hidden layers, proper init
  edge      : 12 hidden layers, POOR init  N(0,1) weights  (far too large)
Training: SGD momentum 0.9, lr 0.01, batch 256, 15 epochs, BCE with pos_weight=10 (fraud is
rare), 3 seeds.
The module is re-used by q7_redesign.py (same data, seeds, budget).
"""
import sys, os, math, time
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
from sklearn.metrics import precision_score, recall_score, f1_score, average_precision_score
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C

WIDTH, LR, MOM, BS, EPOCHS, POSW = 64, 0.01, 0.9, 256, 15, 10.0
ACTS = ["sigmoid", "tanh", "relu"]
COL = {"sigmoid": "#d1495b", "tanh": "#2e86ab", "relu": "#3a9d5d"}
PHI = {"sigmoid": torch.sigmoid, "tanh": torch.tanh, "relu": F.relu}



class MLP(nn.Module):
    def __init__(self, act, depth, init, width=WIDTH, d_in=30, res=False):
        super().__init__()
        self.act, self.depth, self.res = act, depth, res
        self.inp = nn.Linear(d_in, width)
        self.hid = nn.ModuleList([nn.Linear(width, width) for _ in range(depth - 1)])
        self.out = nn.Linear(width, 1)
        self.alpha = 1.0 / math.sqrt(max(depth - 1, 1)) if res else 1.0   # residual branch
scale
        for l in self.layers:
            if init == "xavier":
                nn.init.xavier_normal_(l.weight)
            elif init == "he":
                nn.init.kaiming_normal_(l.weight, nonlinearity="relu")
            elif init == "xavier4":      # used only by Q7: Xavier scaled by 4 to cancel the
sigmoid derivative (<=0.25)
                nn.init.xavier_normal_(l.weight); l.weight.data.mul_(4.0)
            elif init == "bad":
                nn.init.normal_(l.weight, 0.0, 1.0)
            else:
                raise ValueError(init)
            nn.init.zeros_(l.bias)

    @property
    def layers(self):
        return [self.inp] + list(self.hid) + [self.out]

    def forward(self, x, record=False):
        phi = PHI[self.act]
        z = self.inp(x); a = phi(z); h = a
        recs = [(z, a)]
        for lin in self.hid:
            z = lin(h); a = phi(z)
            h = h + self.alpha * a if self.res else a
            recs.append((z, a))
        o = self.out(h).squeeze(-1)
        return (o, recs) if record else o



def layer_grad_norms(model):
    return [l.weight.grad.norm().item() for l in model.layers]



def init_gradient(model, X, y):
    """Full-batch (<=2048 samples) gradient at initialisation."""
    model.zero_grad()
    loss = F.binary_cross_entropy_with_logits(model(X), y, pos_weight=torch.tensor(POSW))
```

```
    loss.backward()
    return layer_grad_norms(model), loss.item()



@torch.no_grad()
def unit_stats(model, X):
    """Per hidden layer: fraction of units that are inactive for EVERY sample (dead ReLU),
    and the mean activation derivative phi'(z) (a saturation indicator for sigmoid/tanh)."""
    _, recs = model(X, record=True)
    dead, dmean = [], []
    for z, a in recs:
        if model.act == "relu":
            dead.append(float(((a > 0).sum(0) == 0).float().mean()))
            dmean.append(float((z > 0).float().mean()))
        elif model.act == "sigmoid":
            dead.append(float("nan")); dmean.append(float((a * (1 - a)).mean()))
        else:
            dead.append(float("nan")); dmean.append(float((1 - a * a).mean()))
    return dead, dmean



@torch.no_grad()
def evaluate(model, X, y):
    o = model(X)
    if not torch.isfinite(o).all():
        o = torch.nan_to_num(o, nan=-1e3, posinf=1e3, neginf=-1e3)
        diverged = True
    else:
        diverged = False
    p = (o > 0).float()
    yn, pn = y.numpy(), p.numpy()
    loss = F.binary_cross_entropy_with_logits(o, y, pos_weight=torch.tensor(POSW)).item()
    return dict(loss=loss, acc=float((p == y).float().mean()),
                prec=float(precision_score(yn, pn, zero_division=0)),
                rec=float(recall_score(yn, pn, zero_division=0)),
                f1=float(f1_score(yn, pn, zero_division=0)),
                auprc=float(average_precision_score(yn, o.numpy())), diverged=diverged)



def run_one(act, depth, init, seed, data, res=False, epochs=EPOCHS, lr=LR, tag=""):
    (Xtr, ytr), (Xva, yva), (Xte, yte) = data
    C.set_seed(seed)
    m = MLP(act, depth, init, res=res)
    opt = torch.optim.SGD(m.parameters(), lr=lr, momentum=MOM)
    gen = torch.Generator().manual_seed(seed)
    g_init, loss_init = init_gradient(m, Xtr[:2048], ytr[:2048])
    dead0, d0 = unit_stats(m, Xva)
    early, curve, step, diverged = [], [], 0, False
    t0 = time.time()
    for ep in range(epochs):
        for xb, yb in C.batches(Xtr, ytr, BS, gen):
            opt.zero_grad()
            loss = F.binary_cross_entropy_with_logits(m(xb), yb, pos_weight=torch.tensor(POSW))
            if not torch.isfinite(loss):
                diverged = True; break
            loss.backward()
            if step < 50:
                early.append(layer_grad_norms(m))
            opt.step(); step += 1
        if diverged:
            break
        tr, va = evaluate(m, Xtr, ytr), evaluate(m, Xva, yva)
        curve.append((ep + 1, tr["loss"], va["loss"], va["acc"], va["rec"]))
    tr, va, te = evaluate(m, Xtr, ytr), evaluate(m, Xva, yva), evaluate(m, Xte, yte)
    dead1, d1 = unit_stats(m, Xva)
    g_final = None
    if not diverged:
        g_final, _ = init_gradient(m, Xtr[:2048], ytr[:2048])
    return dict(act=act, depth=depth, init=init, res=res, seed=seed, tag=tag, diverged=diverged
or va["diverged"],
```

```
                train_loss=tr["loss"], val_loss=va["loss"], val_acc=va["acc"],
val_prec=va["prec"], val_rec=va["rec"],
                val_f1=va["f1"], val_auprc=va["auprc"], test_acc=te["acc"],
test_prec=te["prec"], test_rec=te["rec"],
                test_f1=te["f1"], loss_init=loss_init,
                g_init=g_init, g_early=np.mean(early, 0).tolist() if early else None,
g_final=g_final,
                dead_init=dead0, dead_final=dead1, dphi_init=d0, dphi_final=d1, curve=curve,
                n_params=sum(p.numel() for p in m.parameters()), secs=time.time() - t0)



def get_data():
    return C.load_fraud()



def depth_sweep(data, log):
    """Gradient ratio |g_layer1| / |g_output| at INITIALISATION as the depth grows (proper
init)."""
    (Xtr, ytr), _, _ = data
    rows = []
    for act, init in [("sigmoid", "xavier"), ("tanh", "xavier"), ("relu", "he")]:
        for depth in [2, 4, 6, 8, 12, 16, 24, 32]:
            rs = []
            for s in range(5):
                C.set_seed(100 + s)
                m = MLP(act, depth, init)
                g, _ = init_gradient(m, Xtr[:2048], ytr[:2048])
                rs.append(g[0] / g[-1])
            rows.append(dict(act=act, depth=depth, ratio_mean=float(np.mean(rs)),
ratio_geo=float(np.exp(np.mean(np.log(rs))))))
            log(f"depth sweep {act:7s} L={depth:2d} |g1|/|g_out| = {np.mean(rs):.3e}")
    return pd.DataFrame(rows)



if __name__ == "__main__":
    log = C.Logger("q1")
    data = get_data()
    (Xtr, ytr) = data[0]
    log(f"Dataset: credit-card fraud (ULB), train {tuple(Xtr.shape)} frauds={int(ytr.sum())}; "
        f"val {tuple(data[1][0].shape)} frauds={int(data[1][1].sum())}; test
{tuple(data[2][0].shape)} frauds={int(data[2][1].sum())}")
    configs = {"shallow": (3, "proper"), "deep": (12, "proper"), "edge": (12, "bad")}
    runs = []
    for cname, (depth, initmode) in configs.items():
        for act in ACTS:
            init = "bad" if initmode == "bad" else ("he" if act == "relu" else "xavier")
            for seed in C.SEEDS:
                r = run_one(act, depth, init, seed, data, tag=cname)
                runs.append(r)
                log(f"{cname:7s} {act:7s} seed={seed} diverged={r['diverged']}
train_loss={r['train_loss']:.4f} "
                    f"val_acc={r['val_acc']:.4f} prec={r['val_prec']:.3f}
rec={r['val_rec']:.3f} auprc={r['val_auprc']:.3f} "
                    f"|g1|_early={r['g_early'][0] if r['g_early'] else float('nan'):.2e} "
                    f"dead_relu_final={np.nanmean(r['dead_final']):.3f}")
    # ---- save raw
    import json
    json.dump(runs, open(os.path.join(C.RES, "q1_runs_raw.json"), "w"))
    flat = [{k: v for k, v in r.items() if not isinstance(v, (list, dict))} for r in runs]
    for r, f in zip(runs, flat):
        f["g1_init"], f["g_out_init"] = r["g_init"][0], r["g_init"][-1]
        f["g1_early"] = r["g_early"][0] if r["g_early"] else np.nan
        f["gout_early"] = r["g_early"][-1] if r["g_early"] else np.nan
        f["dead_relu_init_mean"] = float(np.nanmean(r["dead_init"])) if r["act"] == "relu" else
np.nan
        f["dead_relu_final_mean"] = float(np.nanmean(r["dead_final"])) if r["act"] == "relu"
else np.nan
        f["dphi_init_mean"] = float(np.mean(r["dphi_init"])); f["dphi_final_mean"] =
float(np.mean(r["dphi_final"]))
    df = pd.DataFrame(flat); df.to_csv(os.path.join(C.RES, "q1_runs.csv"), index=False)
```

```
    cols = ["train_loss", "val_loss", "val_acc", "val_prec", "val_rec", "val_f1", "val_auprc",
"test_rec", "test_prec",
            "g1_init", "g1_early", "gout_early", "dead_relu_init_mean", "dead_relu_final_mean",
"dphi_init_mean", "dphi_final_mean"]
    summ = df.groupby(["tag", "act"])[cols].agg(["mean", "std"])
    summ["n_diverged"] = df.groupby(["tag", "act"])["diverged"].sum()
    summ.to_csv(os.path.join(C.RES, "q1_summary.csv"))
    log("\n" + df.groupby(["tag", "act"])[["train_loss", "val_acc", "val_prec", "val_rec",
"val_auprc", "g1_early", "dead_relu_final_mean"]].mean().to_string())

    # ---- layer-wise gradient figure (mean over seeds, early training)
    fig, axs = plt.subplots(1, 3, figsize=(13, 3.8), sharey=False)
    for ax, cname in zip(axs, configs):
        for act in ACTS:
            gs = [r["g_early"] for r in runs if r["tag"] == cname and r["act"] == act and
r["g_early"]]
            if not gs:
                continue
            g = np.array(gs).mean(0)
            ax.plot(range(1, len(g) + 1), g, "o-", color=COL[act], label=act, ms=4)
        ax.set_yscale("log"); ax.set_xlabel("layer index (1 = first hidden ... last = output)")
        ax.set_title({"shallow": "Shallow (3 hidden, proper init)", "deep": "Deep (12 hidden,
proper init)",
                      "edge": "Edge (12 hidden, N(0,1) init)"}[cname], fontsize=10)
        ax.grid(alpha=.3, which="both")
    axs[0].set_ylabel("||dL/dW|| (mean of first 50 steps)"); axs[0].legend()
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q1_layer_gradient_norms.png"),
dpi=200); plt.close()

    # ---- training curves (deep + edge)
    fig, axs = plt.subplots(1, 2, figsize=(10, 3.6))
    for ax, cname in zip(axs, ["deep", "edge"]):
        for act in ACTS:
            cs = [r["curve"] for r in runs if r["tag"] == cname and r["act"] == act and
len(r["curve"]) == EPOCHS]
            if cs:
                c = np.array(cs).mean(0)
                ax.plot(c[:, 0], c[:, 1], color=COL[act], label=act)
        ax.set_xlabel("epoch"); ax.set_ylabel("training loss"); ax.set_title(cname);
ax.grid(alpha=.3)
    axs[0].legend(); plt.tight_layout(); plt.savefig(os.path.join(C.FIG,
"q1_training_loss.png"), dpi=200); plt.close()

    # ---- depth sweep
    sw = depth_sweep(data, log); sw.to_csv(os.path.join(C.RES, "q1_depth_sweep.csv"),
index=False)
    plt.figure(figsize=(5.6, 3.8))
    for act in ACTS:
        s = sw[sw.act == act]
        plt.plot(s.depth, s.ratio_geo, "o-", color=COL[act], label=act)
    plt.yscale("log"); plt.xlabel("number of hidden layers"); plt.ylabel("|grad layer 1| /
|grad output layer|")
    plt.grid(alpha=.3, which="both"); plt.legend(); plt.tight_layout()
    plt.savefig(os.path.join(C.FIG, "q1_depth_sweep.png"), dpi=200); plt.close()
    log("done")
```

q2_backprop_numpy.py – Q2

```
"""Q2 - Backpropagation from first principles (NumPy only, NO automatic differentiation).
Hospital 30-day readmission: features = [age, length_of_stay, prior_admissions, n_medications].

Network: x (4) -> Dense(H=8, tanh) -> Dense(1, sigmoid) ; loss = mean binary cross-entropy.
  z1 = x W1 + b1 ;  a1 = tanh(z1) ;  z2 = a1 W2 + b2 ;  p = sigmoid(z2)
  L  = -(1/N) sum[ y log p + (1-y) log(1-p) ]
Chain rule (derived in the report):
  dL/dz2 = (p - y)/N                    (sigmoid + BCE simplification)
  dW2 = a1^T dz2 ;  db2 = sum_n dz2
  da1 = dz2 W2^T ; dz1 = da1 * (1 - a1^2)
  dW1 = x^T dz1  ;  db1 = sum_n dz1
```

```
tanh (smooth) is used in the hidden layer because ReLU has a kink at 0 where finite differences
are unreliable.

Dataset: the hospital data are not public, so a synthetic cohort is generated from a documented
rule (see make_data).
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import common as C

SEED = 0



def make_data(n=3000, seed=SEED):
    rng = np.random.RandomState(seed)
    age = np.clip(rng.normal(65, 14, n), 20, 95)
    los = np.clip(rng.gamma(2.0, 2.5, n), 1, 30)
    prior = rng.poisson(1.5, n).astype(float)
    meds = np.clip(rng.normal(8, 3.5, n), 0, 25).round()
    X = np.stack([age, los, prior, meds], 1)
    Z = (X - X.mean(0)) / X.std(0)
    logit = -1.0 + 0.55 * Z[:, 0] + 0.45 * Z[:, 1] + 0.9 * Z[:, 2] + 0.4 * Z[:, 3] + 0.5 *
np.tanh(1.5 * Z[:, 2] * Z[:, 3])
    y = (rng.rand(n) < 1 / (1 + np.exp(-logit))).astype(float)
    return Z, y



def init_params(H=8, d=4, seed=SEED):
    r = np.random.RandomState(seed)
    return dict(W1=r.randn(d, H) * np.sqrt(1.0 / d), b1=np.zeros(H), W2=r.randn(H, 1) *
np.sqrt(1.0 / H), b2=np.zeros(1))



def sigmoid(z):
    return 1 / (1 + np.exp(-z))



def forward(P, X):
    z1 = X @ P["W1"] + P["b1"]; a1 = np.tanh(z1)
    z2 = a1 @ P["W2"] + P["b2"]; p = sigmoid(z2)
    return z1, a1, z2, p



def loss_fn(P, X, y):
    _, _, _, p = forward(P, X)
    p = np.clip(p[:, 0], 1e-12, 1 - 1e-12)
    return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))



def backward(P, X, y, bug=None):
    """Analytical gradients.  `bug` injects a deliberate error:
         'no_tanh_derivative' : forgets the (1 - a1^2) factor  -> dz1 = da1
         'transpose_W2'       : uses W2 (reshaped) instead of W2^T when propagating to the
hidden layer
         'missing_1_over_N'   : forgets the 1/N of the mean (affects ALL groups)
    """
    N = X.shape[0]
    z1, a1, z2, p = forward(P, X)
    dz2 = (p - y[:, None]) / N                      # (N,1)
    dW2 = a1.T @ dz2; db2 = dz2.sum(0)
    if bug == "transpose_W2":
        da1 = dz2 @ P["W2"].T[:, ::-1]              # hidden units wired in reversed order (a
mis-indexed / mis-transposed W2)
```

```
    else:
        da1 = dz2 @ P["W2"].T                          # (N,H)
    dz1 = da1 if bug == "no_tanh_derivative" else da1 * (1 - a1 ** 2)
    dW1 = X.T @ dz1; db1 = dz1.sum(0)
    g = dict(W1=dW1, b1=db1, W2=dW2, b2=db2)
    if bug == "missing_1_over_N":
        g = {k: v * N for k, v in g.items()}
    return g



def numerical_grad(P, X, y, eps):
    """Central finite differences (L(theta+eps) - L(theta-eps)) / (2 eps), one scalar parameter
at a time."""
    g = {}
    for k in P:
        gk = np.zeros_like(P[k]); it = np.nditer(P[k], flags=["multi_index"])
        for _ in it:
            idx = it.multi_index; old = P[k][idx]
            P[k][idx] = old + eps; lp = loss_fn(P, X, y)
            P[k][idx] = old - eps; lm = loss_fn(P, X, y)
            P[k][idx] = old; gk[idx] = (lp - lm) / (2 * eps)
        g[k] = gk
    return g



def rel_err(a, n):
    return float(np.linalg.norm(a - n) / (np.linalg.norm(a) + np.linalg.norm(n) + 1e-300))



def draw_graph(path):
    fig, ax = plt.subplots(figsize=(11, 3.9)); ax.axis("off"); ax.set_xlim(0, 11);
ax.set_ylim(0, 4)
    def box(x, y, t, c="#dfe9f5", w=1.15, h=0.7):
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.03", fc=c,
ec="#33506b", lw=1.2))
        ax.text(x + w / 2, y + h / 2, t, ha="center", va="center", fontsize=9)
    def arr(x1, y1, x2, y2, c="#33506b", ls="-"):
        ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>", mutation_scale=11,
color=c, lw=1.2, ls=ls))
    ys = 2.6
    box(0.1, ys, "x\n(N×4)", "#fff2cc"); box(0.1, 1.2, "W1 (4×8)", "#e2f0d9"); box(1.7, 0.2,
"b1 (8)", "#e2f0d9")
    box(1.7, ys, "z1 = xW1+b1"); box(3.4, ys, "a1 = tanh(z1)"); box(5.1, ys, "z2 = a1W2+b2")
    box(5.1, 1.2, "W2 (8×1)", "#e2f0d9"); box(6.8, 0.2, "b2 (1)", "#e2f0d9"); box(6.8, ys, "p =
σ(z2)")
    box(8.5, ys, "L = BCE(p,y)", "#f8cbad"); box(8.5, 1.2, "y (labels)", "#fff2cc")
    for a, b in [(1.25, 1.7), (2.85, 3.4), (4.55, 5.1), (6.25, 6.8), (7.95, 8.5)]:
        arr(a, ys + .35, b, ys + .35)
    arr(0.7, 1.9, 1.9, ys); arr(2.3, 0.9, 2.3, ys); arr(5.7, 1.9, 5.7, ys); arr(7.4, 0.9, 7.4,
ys); arr(9.1, 1.9, 9.1, ys)
    ax.text(5.5, 3.65, "forward pass  →", fontsize=9, color="#33506b", ha="center")
    ax.text(5.5, 3.4, "backward pass ←  dL/dz2 = (p−y)/N  →  dW2 = a1ᵀ dz2  →  da1 = dz2 W2ᵀ  →
dz1 = da1 ⊙(1−a1²)  →  dW1 = xᵀ dz1",
            fontsize=8, color="#b03a2e", ha="center")
    plt.tight_layout(); plt.savefig(path, dpi=200); plt.close()



if __name__ == "__main__":
    log = C.Logger("q2")
    X, y = make_data()
    n = len(y); ntr, nva = int(0.7 * n), int(0.15 * n)
    Xtr, ytr, Xva, yva, Xte, yte = X[:ntr], y[:ntr], X[ntr:ntr + nva], y[ntr:ntr + nva], X[ntr
+ nva:], y[ntr + nva:]
    log(f"Dataset: synthetic hospital readmission cohort, n={n} (train {ntr} / val {nva} / test
{n-ntr-nva}); readmission rate={y.mean():.3f}; seed={SEED}")
    draw_graph(os.path.join(C.FIG, "q2_computational_graph.png"))

    # ---- training with analytical gradients (batch gradient descent)
    P = init_params(); lr, epochs = 0.5, 400; hist = []
    for ep in range(epochs + 1):
```

```
        if ep % 20 == 0:
            acc_v = float(((forward(P, Xva)[3][:, 0] > .5) == yva).mean())
            hist.append((ep, loss_fn(P, Xtr, ytr), loss_fn(P, Xva, yva), acc_v))
        g = backward(P, Xtr, ytr)
        for k in P:
            P[k] -= lr * g[k]
    te_acc = float(((forward(P, Xte)[3][:, 0] > .5) == yte).mean()); base = max(yte.mean(), 1 -
yte.mean())
    log(f"trained {epochs} epochs lr={lr}: train loss {hist[-1][1]:.4f} val loss {hist[-
1][2]:.4f} val acc {hist[-1][3]:.4f} test acc {te_acc:.4f} (majority baseline {base:.4f})")
    pd.DataFrame(hist, columns=["epoch", "train_loss", "val_loss",
"val_acc"]).to_csv(os.path.join(C.RES, "q2_training_history.csv"), index=False)
    plt.figure(figsize=(5.4, 3.4)); h = np.array(hist)
    plt.plot(h[:, 0], h[:, 1], label="train"); plt.plot(h[:, 0], h[:, 2], label="validation")
    plt.xlabel("epoch"); plt.ylabel("BCE loss"); plt.grid(alpha=.3); plt.legend();
plt.tight_layout()
    plt.savefig(os.path.join(C.FIG, "q2_training_curve.png"), dpi=200); plt.close()

    # ---- gradient checking on (a) the random init, (b) the trained parameters, with several
eps
    rows = []
    sub = slice(0, 200)  # 200 samples is enough and keeps finite differences cheap
    for state, PP in [("init", init_params()), ("trained", {k: v.copy() for k, v in
P.items()})]:
        ga = backward(PP, Xtr[sub], ytr[sub])
        for eps in [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8]:
            gn = numerical_grad(PP, Xtr[sub], ytr[sub], eps)
            for k in ["W1", "b1", "W2", "b2"]:
                rows.append(dict(state=state, bug="none", eps=eps, group=k,
rel_error=rel_err(ga[k], gn[k])))
    df = pd.DataFrame(rows); df.to_csv(os.path.join(C.RES, "q2_gradcheck_correct.csv"),
index=False)
    log("\nGradient check (correct implementation), relative error ||ga-
gn||/(||ga||+||gn||):\n" +
        df[df.state == "init"].pivot(index="eps", columns="group",
values="rel_error").to_string(float_format=lambda v: f"{v:.2e}"))

    # ---- deliberate bugs
    PP = init_params(); eps = 1e-5
    gn = numerical_grad(PP, Xtr[sub], ytr[sub], eps); brow = []
    for bug in ["no_tanh_derivative", "transpose_W2", "missing_1_over_N"]:
        ga = backward(PP, Xtr[sub], ytr[sub], bug=bug)
        for k in ["W1", "b1", "W2", "b2"]:
            brow.append(dict(bug=bug, group=k, rel_error=rel_err(ga[k], gn[k])))
    bd = pd.DataFrame(brow); bd.to_csv(os.path.join(C.RES, "q2_gradcheck_bugs.csv"),
index=False)
    log("\nGradient check with injected bugs (eps=1e-5):\n" + bd.pivot(index="bug",
columns="group", values="rel_error")[["W1", "b1", "W2", "b2"]].to_string(float_format=lambda v:
f"{v:.2e}"))

    # ---- figure: relative error vs eps
    plt.figure(figsize=(6, 3.8))
    d0 = df[df.state == "init"]
    for k, c in zip(["W1", "b1", "W2", "b2"], ["#2e86ab", "#3a9d5d", "#d1495b", "#e09f3e"]):
        s = d0[d0.group == k]; plt.loglog(s.eps, s.rel_error, "o-", color=c, label=k)
    plt.axhline(1e-7, color="k", ls="--", lw=.8); plt.text(1.2e-8, 1.4e-7, "1e-7", fontsize=8)
    plt.xlabel("finite-difference step ε"); plt.ylabel("relative error"); plt.grid(alpha=.3,
which="both"); plt.legend()
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q2_gradcheck_eps.png"), dpi=200);
plt.close()
    log("done")
```

q3_regularization.py – Q3

```
"""Q3 - Engineer overfitting, then control it (regularisation).
Stand-in dataset for the plant-leaf problem: Fashion-MNIST (10 classes, 28x28) because no plant
images are
reachable from this environment; the experiment is about training-set size x capacity, which is
data independent.
```

```
CNN (~0.42 M parameters): conv3x3(1->32)-ReLU-pool-conv3x3(32->64)-ReLU-pool-FC(3136->128)-
ReLU-FC(128->10)

Conditions (validation = 1,000 clean images disjoint from training; test = official 10,000):
  A  generalises well : 20,000 training images, 6 epochs
  B  severe overfitting: 300 training images, 60 epochs (parameters/samples ~ 1400)
  C  label noise       : 3,000 training images with 30% symmetric label noise (validation/test
labels clean), 30 epochs
(A first attempt with A = 10,000 images x 15 epochs showed an 8-10 point train-validation gap -
> not a 'good' condition; its log is kept in
 logs/q3_pilot_first_attempt.log and it was replaced by the 20,000-image / 6-epoch version.)
Run:  python q3_regularization.py worker K N   (executes jobs with index % N == K; resumable,
one JSON per job in results/q3_parts/)
      python q3_regularization.py report       (aggregates the parts into CSV + figures)
Regularisers on condition B: L2 weight decay, dropout, early stopping, data augmentation,
batch-norm, label smoothing,
and three combinations.  Adam lr 1e-3, 3 seeds.  Early stopping = restore the weights of the
epoch with minimum validation loss.
"""
import sys, os, copy, time, json
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C

BG = (0 - C.FM_MEAN) / C.FM_STD   # normalised value of a black pixel



class CNN(nn.Module):
    def __init__(self, drop=0.0, bn=False):
        super().__init__()
        self.c1, self.c2 = nn.Conv2d(1, 32, 3, padding=1), nn.Conv2d(32, 64, 3, padding=1)
        self.f1, self.f2 = nn.Linear(64 * 7 * 7, 128), nn.Linear(128, 10)
        self.b1, self.b2, self.b3 = (nn.BatchNorm2d(32), nn.BatchNorm2d(64),
nn.BatchNorm1d(128)) if bn else (nn.Identity(),) * 3
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.b1(self.c1(x))), 2)
        x = F.max_pool2d(F.relu(self.b2(self.c2(x))), 2)
        x = self.drop(F.relu(self.b3(self.f1(x.flatten(1)))))
        return self.f2(x)



def weight_norm(m):
    return float(torch.sqrt(sum((p ** 2).sum() for n, p in m.named_parameters() if p.dim() >
1)))



def augment(x, gen):
    """random translation (+-2 px) and horizontal flip."""
    B = x.shape[0]
    pad = F.pad(x, (2, 2, 2, 2), value=BG)
    d = torch.randint(0, 5, (B, 2), generator=gen)
    out = torch.stack([pad[i, :, d[i, 0]:d[i, 0] + 28, d[i, 1]:d[i, 1] + 28] for i in
range(B)])
    flip = torch.rand(B, generator=gen) < 0.5
    out[flip] = out[flip].flip(3)
    return out



@torch.no_grad()
def evaluate(m, X, y, bs=1000):
    m.eval(); L = 0.0; A = 0
    for i in range(0, len(X), bs):
```

```
        o = m(X[i:i + bs]); L += F.cross_entropy(o, y[i:i + bs], reduction="sum").item(); A += (o.argmax(1) == y[i:i + bs]).sum().item()
    return L / len(X), A / len(X)



def noisy_labels(y, frac, seed):
    r = np.random.RandomState(seed + 99); y2 = y.clone(); idx = r.permutation(len(y))[:int(frac * len(y))]
    for i in idx:
        y2[i] = (y[i] + r.randint(1, 10)) % 10
    return y2



def train(cond, cfg, seed, data, epochs, bs, ytrain_clean=None):
    Xtr, ytr, Xva, yva, Xte, yte = data
    C.set_seed(seed)
    m = CNN(cfg.get("drop", 0.0), cfg.get("bn", False))
    opt = torch.optim.Adam(m.parameters(), lr=1e-3, weight_decay=cfg.get("wd", 0.0))
    gen = torch.Generator().manual_seed(seed)
    ls = cfg.get("ls", 0.0)
    rows, best, best_state = [], 1e9, None
    for ep in range(1, epochs + 1):
        m.train()
        perm = torch.randperm(len(Xtr), generator=gen)
        for i in range(0, len(Xtr), bs):
            j = perm[i:i + bs]
            if len(j) < 2:
                continue
            xb = augment(Xtr[j], gen) if cfg.get("aug") else Xtr[j]
            opt.zero_grad(); F.cross_entropy(m(xb), ytr[j], label_smoothing=ls).backward(); opt.step()
        trl, tra = evaluate(m, Xtr, ytr); val, vaa = evaluate(m, Xva, yva)
        row = dict(epoch=ep, train_loss=trl, train_acc=tra, val_loss=val, val_acc=vaa, wnorm=weight_norm(m))
        if ytrain_clean is not None:
            row["train_acc_clean"] = evaluate(m, Xtr, ytrain_clean)[1]
        rows.append(row)
        if val < best:
            best, best_state = val, copy.deepcopy(m.state_dict())
    te_last = evaluate(m, Xte, yte)[1]
    m.load_state_dict(best_state); te_best = evaluate(m, Xte, yte)[1]
    cur = pd.DataFrame(rows)
    ib = int(cur.val_loss.idxmin())
    last = cur.iloc[-1]
    summ = dict(cond=cond, cfg=cfg["name"], seed=seed, epochs=epochs,
                train_acc_final=last.train_acc, val_acc_final=last.val_acc, gap_final=last.train_acc - last.val_acc,
                train_loss_final=last.train_loss, val_loss_final=last.val_loss, wnorm_final=last.wnorm, test_acc_final=te_last,
                best_epoch=int(cur.epoch[ib]), val_loss_min=float(cur.val_loss[ib]), train_acc_at_best=float(cur.train_acc[ib]),
                val_acc_at_best=float(cur.val_acc[ib]), gap_at_best=float(cur.train_acc[ib] - cur.val_acc[ib]),
                wnorm_at_best=float(cur.wnorm[ib]), test_acc_at_best=te_best)
    if ytrain_clean is not None:
        summ["train_acc_clean_final"] = float(last.train_acc_clean)
    return summ, cur




SPECS = {"A": dict(n=20000, ep=6, bs=64, noise=0.0), "B": dict(n=300, ep=60, bs=32, noise=0.0), "C": dict(n=3000, ep=30, bs=64, noise=0.3)}
NVAL = 1000
CFGS = [dict(name="baseline"),
        dict(name="L2 1e-4", wd=1e-4), dict(name="L2 1e-3", wd=1e-3), dict(name="L2 1e-2", wd=1e-2),
        dict(name="Dropout 0.3", drop=0.3), dict(name="Dropout 0.5", drop=0.5),
        dict(name="Augmentation", aug=True), dict(name="BatchNorm", bn=True), dict(name="Label smoothing 0.1", ls=0.1),
        dict(name="Aug + Dropout 0.5", aug=True, drop=0.5), dict(name="L2 1e-3 + Dropout 0.5", wd=1e-3, drop=0.5),
```

```
        dict(name="Aug + Label smoothing", aug=True, ls=0.1)]
PARTS = os.path.join(C.RES, "q3_parts")



def job_list():
    jobs = []
    for seed in C.SEEDS:
        jobs += [("A", "baseline", seed), ("C", "baseline", seed), ("B", "baseline", seed)]
    for cfg in CFGS[1:]:
        for seed in C.SEEDS:
            jobs.append(("B", cfg["name"], seed))
    return jobs



def run_job(cond, cfgname, seed, log):
    fn = os.path.join(PARTS, f"{cond}__{cfgname.replace(' ', '_').replace('+',
'plus')}__{seed}.json")
    if os.path.exists(fn):
        return
    s = SPECS[cond]; cfg = next(c for c in CFGS if c["name"] == cfgname)
    Xtr, ytr, Xva, yva, Xte, yte = C.fmnist_split(s["n"], NVAL, seed=seed)
    clean = ytr
    if s["noise"] > 0:
        ytr = noisy_labels(ytr, s["noise"], seed)
    r, cur = train(cond, cfg, seed, (Xtr, ytr, Xva, yva, Xte, yte), s["ep"], s["bs"],
ytrain_clean=clean if s["noise"] > 0 else None)
    json.dump(dict(summary=r, curve=cur.to_dict("list")), open(fn, "w"))
    log(f"{cond} {cfgname:24s} seed={seed}: train={r['train_acc_final']:.3f}
val={r['val_acc_final']:.3f} gap={r['gap_final']:.3f} "
        f"best_ep={r['best_epoch']} gap@best={r['gap_at_best']:.3f}
val@best={r['val_acc_at_best']:.3f} wnorm={r['wnorm_final']:.1f}
test={r['test_acc_final']:.3f}")



def report(log):
    summ, curves = [], {}
    for f in sorted(os.listdir(PARTS)):
        d = json.load(open(os.path.join(PARTS, f))); r = d["summary"]
        key = f"{r['cond']}|{r['cfg']}|{r['seed']}"
        r["regulariser_run"] = r["cond"] == "B"
        summ.append(r); curves[key] = d["curve"]
    df = pd.DataFrame(summ); df.to_csv(os.path.join(C.RES, "q3_runs.csv"), index=False)
    json.dump(curves, open(os.path.join(C.RES, "q3_curves.json"), "w"))
    base = df[df.cfg == "baseline"].groupby("cond").mean(numeric_only=True)
    log("\nbaseline conditions (mean of 3 seeds):\n" + base[["train_acc_final",
"val_acc_final", "gap_final", "best_epoch", "gap_at_best", "val_acc_at_best", "wnorm_final",
"test_acc_final", "test_acc_at_best"]].round(3).to_string())


    def mean_curve(prefix, field):
        return np.mean([np.array(v[field]) for k, v in curves.items() if k.startswith(prefix +
"|")], 0)
    fig, axs = plt.subplots(2, 3, figsize=(13, 6.2))
    for j, (cond, ttl) in enumerate([("A", "A: 20,000 images (good)"), ("B", "B: 300 images
(overfit)"), ("C", "C: 3,000 imgs + 30% label noise")]):
        ne = SPECS[cond]["ep"]
        for field, c, lab in [("train_loss", "#2e86ab", "train"), ("val_loss", "#d1495b",
"validation")]:
            axs[0, j].plot(range(1, ne + 1), mean_curve(f"{cond}|baseline", field), color=c,
label=lab)
        axs[0, j].set_title(ttl, fontsize=10); axs[0, j].set_xlabel("epoch"); axs[0,
j].set_ylabel("cross-entropy loss"); axs[0, j].grid(alpha=.3)
        for field, c, lab in [("train_acc", "#2e86ab", "train"), ("val_acc", "#d1495b",
"validation")]:
            axs[1, j].plot(range(1, ne + 1), mean_curve(f"{cond}|baseline", field), color=c,
label=lab)
        if cond == "C":
            axs[1, j].plot(range(1, ne + 1), mean_curve("C|baseline", "train_acc_clean"), "--",
color="#3a9d5d", label="train (true labels)")
        axs[1, j].set_xlabel("epoch"); axs[1, j].set_ylabel("accuracy"); axs[1,
j].grid(alpha=.3)
```

```
    axs[0, 0].legend(); axs[1, 2].legend(fontsize=8); plt.tight_layout();
plt.savefig(os.path.join(C.FIG, "q3_conditions_curves.png"), dpi=200); plt.close()

    names = [c["name"] for c in CFGS]
    fig, axs = plt.subplots(1, 2, figsize=(13, 4.2)); cm = plt.get_cmap("tab20")
    for i, n in enumerate(names):
        axs[0].plot(range(1, 61), mean_curve(f"B|{n}", "val_loss"), color=cm(i), label=n,
lw=1.6)
        axs[1].plot(range(1, 61), mean_curve(f"B|{n}", "train_acc") - mean_curve(f"B|{n}",
"val_acc"), color=cm(i), label=n, lw=1.6)
    axs[0].set_ylim(0.3, 3); axs[0].set_xlabel("epoch"); axs[0].set_ylabel("validation loss");
axs[0].grid(alpha=.3)
    axs[1].set_xlabel("epoch"); axs[1].set_ylabel("train acc - val acc");
axs[1].grid(alpha=.3); axs[1].legend(fontsize=7, ncol=2)
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q3_regulariser_curves.png"), dpi=200);
plt.close()

    reg = df[df.regulariser_run].groupby("cfg", sort=False).mean(numeric_only=True).loc[names]
    regs = df[df.regulariser_run].groupby("cfg", sort=False).std(numeric_only=True).loc[names]
    reg.to_csv(os.path.join(C.RES, "q3_regulariser_mean.csv")); regs.to_csv(os.path.join(C.RES,
"q3_regulariser_std.csv"))
    fig, axs = plt.subplots(1, 3, figsize=(14, 4.4)); y = np.arange(len(reg))
    axs[0].barh(y, reg.gap_final, xerr=regs.gap_final, color="#2e86ab");
axs[0].set_title("train-val accuracy gap (final epoch)", fontsize=10)
    axs[1].barh(y, reg.val_acc_final, xerr=regs.val_acc_final, color="#3a9d5d");
axs[1].set_xlim(0.5, 0.9); axs[1].set_title("validation accuracy (final epoch)", fontsize=10)
    axs[2].barh(y, reg.wnorm_final, color="#e09f3e"); axs[2].set_title("weight norm ||W||₂
(final epoch)", fontsize=10)
    for a in axs:
        a.set_yticks(y); a.set_yticklabels(reg.index if a is axs[0] else [], fontsize=8);
a.invert_yaxis(); a.grid(alpha=.3, axis="x")
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q3_regulariser_bars.png"), dpi=200);
plt.close()
    log("\n" + reg[["train_acc_final", "val_acc_final", "gap_final", "best_epoch",
"gap_at_best", "val_acc_at_best", "wnorm_final", "test_acc_final",
"test_acc_at_best"]].round(3).to_string())



if __name__ == "__main__":
    os.makedirs(PARTS, exist_ok=True)
    mode = sys.argv[1]
    if mode == "worker":
        k, n = int(sys.argv[2]), int(sys.argv[3]); log = C.Logger(f"q3_worker{k}")
        for i, (cond, cfgname, seed) in enumerate(job_list()):
            if i % n == k:
                run_job(cond, cfgname, seed, log)
        log("worker done")
    else:
        report(C.Logger("q3_report")); print("report written")
```

q4_optimizers.py – Q4

```
"""Q4 - Optimisation for training deep models: which optimiser, under which conditions?
Electricity-demand forecasting (next-hour load) with a deep feedforward network (13 -> 64 -> 64
-> 64 -> 1, ReLU).

DATA: no public load data reachable from this environment, so a documented SYNTHETIC hourly
series is generated
(3 years = 26,280 h): daily double-peak profile x weekday factor + annual seasonality +
temperature-driven
heating/cooling demand + holidays + AR(1) fluctuation + white measurement noise (sd = 10% of
the signal sd).
Features at hour t (13): sin/cos hour, sin/cos weekday, sin/cos month, weekend flag, holiday
flag, temperature,
humidity, load(t), load(t-23), load(t-167)  ->  target load(t+1).  Chronological split: 2 years
train (17,520 samples
after the lag warm-up), 6 months validation, 6 months test.  Inputs and target standardised
with training statistics.
Loss = MSE in standardised units, so validation MSE = 1 - R^2.
```

```
Controlled grid: 4 optimisers x 3 learning rates {1e-3, 1e-2, 1e-1} x 2 batch sizes {64, 512} x
3 seeds, 25 epochs, same init per seed.
"""
import sys, os, json, time
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C

OPTS = ["SGD", "Momentum", "RMSProp", "Adam"]
LRS = [1e-3, 1e-2, 1e-1]
BSS = [64, 512]
EPOCHS = 25
BUDGET = 10          # nightly budget: at most 10 epochs
TARGET = 0.05        # target validation MSE (standardised) = R^2 of 0.95 (noise floor ~0.01).
A first guess of 0.03 was
                     # abandoned after a 4-run pilot showed that even Adam only reached 0.037
in 10 epochs (too strict).
COL = {"SGD": "#d1495b", "Momentum": "#e09f3e", "RMSProp": "#2e86ab", "Adam": "#3a9d5d"}



def make_series(seed=2024):
    rng = np.random.RandomState(seed)
    n = 3 * 365 * 24
    t = np.arange(n); hour = t % 24; dow = (t // 24) % 7; doy = (t // 24) % 365; month = (doy
// 30.42).astype(int) % 12
    holiday = np.zeros(n); hol_days = rng.choice(np.arange(3 * 365), 33, replace=False)
    for d in hol_days:
        holiday[d * 24:(d + 1) * 24] = 1
    # weather
    ar = np.zeros(n)
    for i in range(1, n):
        ar[i] = 0.97 * ar[i - 1] + rng.randn() * 0.6
    temp = 18 - 10 * np.cos(2 * np.pi * (doy - 15) / 365) - 4 * np.cos(2 * np.pi * (hour - 3) /
24) + ar
    hum = np.clip(60 + 15 * np.sin(2 * np.pi * (hour + 6) / 24) - 0.8 * (temp - 18) +
rng.randn(n) * 4, 20, 100)
    # load
    daily = 0.6 * np.exp(-0.5 * ((hour - 8) / 2.2) ** 2) + 1.0 * np.exp(-0.5 * ((hour - 19) /
2.8) ** 2) + 0.35 * np.sin(np.pi * (hour - 4) / 20).clip(0)
    week = np.where(dow >= 5, 0.82, 1.0) * np.where(holiday > 0, 0.80, 1.0)
    hvac = 0.012 * (temp - 18) ** 2 * (1 + 0.5 * (temp > 18)) + 0.003 * hum * (temp > 22)
    fluct = np.zeros(n)
    for i in range(1, n):
        fluct[i] = 0.9 * fluct[i - 1] + rng.randn() * 0.05
    signal = (1.0 + daily) * week * (1 + 0.08 * np.cos(2 * np.pi * (doy - 20) / 365)) + hvac +
fluct
    load = signal + rng.randn(n) * 0.10 * signal.std()
    mnt = np.stack([np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24), np.sin(2 *
np.pi * dow / 7), np.cos(2 * np.pi * dow / 7),
                    np.sin(2 * np.pi * month / 12), np.cos(2 * np.pi * month / 12), (dow >=
5).astype(float), holiday, temp, hum], 1)
    s = 167
    idx = np.arange(s, n - 1)
    X = np.concatenate([mnt[idx], load[idx, None], load[idx - 23 + 0, None] if False else
load[idx - 23, None], load[idx - 167, None]], 1)
    y = load[idx + 1]
    ntr, nva = 2 * 365 * 24 - s, 182 * 24
    sl = lambda a, b: slice(a, b)
    mu, sd = X[:ntr].mean(0), X[:ntr].std(0); ym, ys = y[:ntr].mean(), y[:ntr].std()
    f = lambda a: torch.tensor(a, dtype=torch.float32)
    Z = lambda a, b: (f((X[a:b] - mu) / sd), f(((y[a:b] - ym) / ys)))
    floor = (0.10 * signal.std()) ** 2 / ys ** 2
    return Z(0, ntr), Z(ntr, ntr + nva), Z(ntr + nva, len(y)), float(ys), float(floor)



def make_model(seed):
    C.set_seed(seed)
```

```
    m = nn.Sequential(nn.Linear(13, 64), nn.ReLU(), nn.Linear(64, 64), nn.ReLU(), nn.Linear(64,
64), nn.ReLU(), nn.Linear(64, 1))
    for l in m:
        if isinstance(l, nn.Linear):
            nn.init.kaiming_normal_(l.weight, nonlinearity="relu"); nn.init.zeros_(l.bias)
    return m



def make_opt(name, params, lr):
    if name == "SGD":
        return torch.optim.SGD(params, lr=lr)
    if name == "Momentum":
        return torch.optim.SGD(params, lr=lr, momentum=0.9)
    if name == "RMSProp":
        return torch.optim.RMSprop(params, lr=lr, alpha=0.9, eps=1e-8)
    return torch.optim.Adam(params, lr=lr, betas=(0.9, 0.999), eps=1e-8)



@torch.no_grad()
def val_metrics(m, X, y, ys):
    p = m(X).squeeze(-1); d = p - y
    mse = float((d ** 2).mean())
    return mse, float(torch.sqrt((d ** 2).mean()) * ys), float(d.abs().mean() * ys)



def run(optname, lr, bs, seed, data, ys, epochs=EPOCHS):
    (Xtr, ytr), (Xva, yva), _ = data
    m = make_model(seed); opt = make_opt(optname, m.parameters(), lr); gen =
torch.Generator().manual_seed(seed)
    mse0 = val_metrics(m, Xva, yva, ys)[0]
    curve, t0, tt = [mse0], time.perf_counter(), []
    diverged = False
    for ep in range(epochs):
        for xb, yb in C.batches(Xtr, ytr, bs, gen):
            opt.zero_grad(); loss = F.mse_loss(m(xb).squeeze(-1), yb)
            if not torch.isfinite(loss):
                diverged = True; break
            loss.backward(); opt.step()
        if diverged:
            curve += [float("nan")] * (epochs - ep); break
        mse, rmse, mae = val_metrics(m, Xva, yva, ys); curve.append(mse);
tt.append(time.perf_counter() - t0)
        if not np.isfinite(mse):
            diverged = True; curve += [float("nan")] * (epochs - ep - 1); break
    c = np.array(curve, float)
    reached = np.where(c[1:] <= TARGET)[0]
    ep_target = int(reached[0] + 1) if len(reached) else np.nan
    cc = c[1:][np.isfinite(c[1:])]
    ups = int((np.diff(cc) > 0.1 * cc[:-1]).sum()) if len(cc) > 1 else 0
    final_mse = float(c[-1]) if np.isfinite(c[-1]) else np.nan
    ok = not diverged and np.isfinite(final_mse) and final_mse < mse0
    return dict(opt=optname, lr=lr, bs=bs, seed=seed, diverged=bool(diverged or not ok),
epochs_to_target=ep_target,
                reached_in_budget=bool(np.isfinite(ep_target) and ep_target <= BUDGET),
                final_val_mse=final_mse, final_val_rmse=float(np.sqrt(final_mse) * ys) if
np.isfinite(final_mse) else np.nan,
                final_val_mae=val_metrics(m, Xva, yva, ys)[2] if ok else np.nan,
best_val_mse=float(np.nanmin(c[1:])) if len(cc) else np.nan,
                n_loss_spikes=ups, secs_per_epoch=float(np.mean(np.diff([0] + tt))) if tt else
np.nan, curve=curve)



def step_diagnostics(optname, lr, bs, data, nsteps=200, seed=0):
    """Measures, over the first nsteps updates: step length ||dtheta||, cosine(dtheta, -grad)
(direction),
    and the spread (coefficient of variation) of per-parameter |dtheta_i|/|g_i| (per-parameter
adaptation)."""
    (Xtr, ytr), _, _ = data
    m = make_model(seed); params = [p for p in m.parameters()]; opt = make_opt(optname, params,
lr); gen = torch.Generator().manual_seed(seed)
```

```
    L, cs, cv = [], [], []; it = 0
    for xb, yb in C.batches(Xtr, ytr, bs, gen):
        opt.zero_grad(); F.mse_loss(m(xb).squeeze(-1), yb).backward()
        g = torch.cat([p.grad.flatten() for p in params]); old =
torch.cat([p.detach().flatten() for p in params])
        opt.step(); d = torch.cat([p.detach().flatten() for p in params]) - old
        L.append(float(d.norm())); cs.append(float(F.cosine_similarity(d, -g, dim=0)))
        r = (d.abs() / (g.abs() + 1e-12)); r = r[g.abs() > 1e-6]
        cv.append(float(r.std() / (r.mean() + 1e-12)))
        it += 1
        if it >= nsteps:
            break
    return dict(opt=optname, lr=lr, step_len=float(np.mean(L)),
cos_to_neg_grad=float(np.mean(cs)), adapt_cv=float(np.mean(cv)))



if __name__ == "__main__":
    log = C.Logger("q4")
    data = make_series(); ys, floor = data[3], data[4]; data = data[:3]
    log(f"Data: synthetic hourly load; train {tuple(data[0][0].shape)} val
{tuple(data[1][0].shape)} test {tuple(data[2][0].shape)}; "
        f"target std={ys:.3f} (load units); noise-floor val MSE ~ {floor:.4f}; TARGET val MSE =
{TARGET}; nightly BUDGET = {BUDGET} epochs")
    # reference: ridge regression on the same features
    X, y = data[0]; Xv, yv = data[1]
    w = torch.linalg.solve(X.T @ X + 1e-3 * torch.eye(13), X.T @ y); log(f"reference linear
regression val MSE = {float(((Xv @ w - yv) ** 2).mean()):.4f}")
    rows = []
    for o in OPTS:
        for lr in LRS:
            for bs in BSS:
                for seed in C.SEEDS:
                    r = run(o, lr, bs, seed, data, ys); rows.append(r)
                log(f"{o:8s} lr={lr:<6g} bs={bs:3d}: " + " | ".join(f"seed{r['seed']} ep-
>target={r['epochs_to_target']} final_mse={r['final_val_mse']:.4f} div={r['diverged']}"
                                                                    for r in rows[-3:]))
    df = pd.DataFrame([{k: v for k, v in r.items() if k != "curve"} for r in rows]);
df.to_csv(os.path.join(C.RES, "q4_runs.csv"), index=False)
    json.dump([{k: v for k, v in r.items()} for r in rows], open(os.path.join(C.RES,
"q4_curves.json"), "w"))
    g = df.groupby(["opt", "lr", "bs"], sort=False).agg(reach_rate=("reached_in_budget",
"mean"), ep_target_median=("epochs_to_target", "median"),
        final_mse=("final_val_mse", "mean"), final_rmse=("final_val_rmse", "mean"),
final_mae=("final_val_mae", "mean"), spikes=("n_loss_spikes", "mean"),
        diverged=("diverged", "sum"), sec_per_epoch=("secs_per_epoch", "mean")).reset_index()
    g.to_csv(os.path.join(C.RES, "q4_summary.csv"), index=False)
    log("\n" + g.round(4).to_string())


    # ---- step diagnostics at the best lr per optimiser (bs 64)
    diag = []
    for o in OPTS:
        gg = g[(g.opt == o) & (g.bs == 64)].sort_values(["reach_rate", "final_mse"],
ascending=[False, True]).iloc[0]
        diag.append(step_diagnostics(o, float(gg.lr), 64, data))
    diag = pd.DataFrame(diag); diag.to_csv(os.path.join(C.RES, "q4_step_diagnostics.csv"),
index=False)
    log("\nstep diagnostics (best lr per optimiser, bs=64):\n" + diag.round(4).to_string())


    # ---- figures
    fig, axs = plt.subplots(2, 3, figsize=(13.5, 6.4), sharey=True)
    for i, bs in enumerate(BSS):
        for j, lr in enumerate(LRS):
            ax = axs[i, j]
            for o in OPTS:
                cs = np.array([r["curve"] for r in rows if r["opt"] == o and r["lr"] == lr and
r["bs"] == bs], float)
                with np.errstate(all="ignore"):
                    mu = np.nanmean(cs, 0)
                ax.plot(range(0, EPOCHS + 1), mu, color=COL[o], label=o, lw=1.6)
            ax.axhline(TARGET, color="k", ls="--", lw=.8); ax.axvline(BUDGET, color="gray",
ls=":", lw=.8)
```

```
            ax.set_yscale("log"); ax.set_title(f"lr={lr:g}, batch={bs}", fontsize=10);
ax.grid(alpha=.3, which="both"); ax.set_ylim(0.005, 3)
            if i == 1: ax.set_xlabel("epoch")
            if j == 0: ax.set_ylabel("validation MSE (standardised)")
    axs[0, 0].legend(fontsize=8); plt.tight_layout(); plt.savefig(os.path.join(C.FIG,
"q4_convergence_grid.png"), dpi=200); plt.close()

    fig, axs = plt.subplots(1, 2, figsize=(11, 3.6))
    for ax, bs in zip(axs, BSS):
        mat = np.array([[g[(g.opt == o) & (g.lr == lr) & (g.bs == bs)].reach_rate.iloc[0] for
lr in LRS] for o in OPTS])
        im = ax.imshow(mat, vmin=0, vmax=1, cmap="Greens"); ax.set_xticks(range(3));
ax.set_xticklabels([f"{l:g}" for l in LRS]); ax.set_yticks(range(4)); ax.set_yticklabels(OPTS)
        ax.set_title(f"fraction of seeds reaching target in {BUDGET} epochs (batch {bs})",
fontsize=9); ax.set_xlabel("learning rate")
        for a in range(4):
            for b in range(3):
                e = g[(g.opt == OPTS[a]) & (g.lr == LRS[b]) & (g.bs ==
bs)].ep_target_median.iloc[0]
                ax.text(b, a, f"{mat[a, b]:.2f}\n({'-' if not np.isfinite(e) else int(e)} ep)",
ha="center", va="center", fontsize=8)
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q4_budget_heatmap.png"), dpi=200);
plt.close()
    log("done")
```

q4_post.py – Q4 post-analysis

```
"""Q4 post-analysis (reads results/q4_curves.json written by q4_optimizers.py; no retraining of
the grid).
  * stable reach: validation MSE <= TARGET at some epoch <= BUDGET  AND still <= TARGET at the
end of the budget (rules out 'touched the target then diverged')
  * wall-clock seconds to target
  * cleaner decomposition of an optimiser step into: step length, direction (cosine with -
gradient) and per-parameter adaptation
    (CV of |delta theta_i| divided by CV of |g_i|;  =1 if every parameter moves proportionally
to its own gradient (SGD), <<1 if steps are normalised per parameter)
"""
import sys, os, json
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C
import q4_optimizers as Q

if __name__ == "__main__":
    log = C.Logger("q4_post")
    rows = json.load(open(os.path.join(C.RES, "q4_curves.json")))
    runs = pd.read_csv(os.path.join(C.RES, "q4_runs.csv"))
    summ = pd.read_csv(os.path.join(C.RES, "q4_summary.csv"))
    mse_budget = [r["curve"][Q.BUDGET] for r in rows]
    runs["mse_at_budget"] = mse_budget
    runs["stable_reach"] = runs.reached_in_budget & (runs.mse_at_budget <= Q.TARGET)
    g = runs.groupby(["opt", "lr", "bs"], sort=False).agg(
        stable_rate=("stable_reach", "mean"), reach_rate=("reached_in_budget", "mean"),
        ep_target_median=("epochs_to_target", "median"), mse_at_budget=("mse_at_budget",
"mean"),
        rmse_final=("final_val_rmse", "mean"), mae_final=("final_val_mae", "mean"),
spikes=("n_loss_spikes", "mean"),
        n_diverged=("diverged", "sum"), sec_per_epoch=("secs_per_epoch", "mean")).reset_index()
    g["sec_to_target"] = g.ep_target_median * g.sec_per_epoch
    g.to_csv(os.path.join(C.RES, "q4_summary_stable.csv"), index=False)
    log("\n" + g.round(4).to_string())

    # ---- recommendation by evidence
    rel = g[g.stable_rate == 1.0].sort_values(["ep_target_median", "sec_to_target"])
    log("\nconfigurations that reach AND stay at the target within the 10-epoch budget in 3/3
seeds, fastest first:\n" + rel.round(4).head(12).to_string())
```

```
    byopt = rel.groupby("opt").head(1)
    log("\nbest reliable configuration per optimiser:\n" + byopt.round(4).to_string())
    # robustness: number of (lr, bs) cells per optimiser that are reliable
    log("\nreliable cells per optimiser (out of 6):\n" + g.groupby("opt",
sort=False).stable_rate.apply(lambda s: int((s == 1.0).sum())).to_string())

    # ---- step decomposition
    def diag(optname, lr, bs, nsteps=200, seed=0):
        data = Q.make_series()[:3]; (Xtr, ytr) = data[0]
        m = Q.make_model(seed); params = list(m.parameters()); opt = Q.make_opt(optname,
params, lr); gen = torch.Generator().manual_seed(seed)
        L, cs, ratio = [], [], []; it = 0
        for xb, yb in C.batches(Xtr, ytr, bs, gen):
            opt.zero_grad(); F.mse_loss(m(xb).squeeze(-1), yb).backward()
            gr = torch.cat([p.grad.flatten() for p in params]); old =
torch.cat([p.detach().flatten() for p in params])
            opt.step(); d = torch.cat([p.detach().flatten() for p in params]) - old
            L.append(float(d.norm())); cs.append(float(F.cosine_similarity(d, -gr, dim=0)))
            cv = lambda v: float(v.std() / (v.mean() + 1e-12)); ratio.append(cv(d.abs()) /
cv(gr.abs()))
            it += 1
            if it >= nsteps: break
        return dict(opt=optname, lr=lr, bs=bs, step_len=float(np.mean(L)),
grad_len=float(np.mean([0])), cos_to_neg_grad=float(np.mean(cs)),
cv_ratio=float(np.mean(ratio)))
    ds = []
    for o in Q.OPTS:
        # same nominal lr (0.01) for all, plus each optimiser's best reliable lr for bs=64
        ds.append(diag(o, 0.01, 64))
    ds = pd.DataFrame(ds); ds.to_csv(os.path.join(C.RES, "q4_step_decomposition.csv"),
index=False)
    log("\nstep decomposition at lr=0.01, bs=64 (first 200 updates):\n" +
ds.round(4).to_string())

    # ---- heat-map with stable reach
    fig, axs = plt.subplots(1, 2, figsize=(11.5, 3.7))
    for ax, bs in zip(axs, Q.BSS):
        mat = np.array([[g[(g.opt == o) & (g.lr == lr) & (g.bs == bs)].stable_rate.iloc[0] for
lr in Q.LRS] for o in Q.OPTS])
        ax.imshow(mat, vmin=0, vmax=1, cmap="Greens"); ax.set_xticks(range(3));
ax.set_xticklabels([f"{l:g}" for l in Q.LRS]); ax.set_yticks(range(4));
ax.set_yticklabels(Q.OPTS)
        ax.set_title(f"seeds reaching AND holding target ≤ {Q.TARGET} within {Q.BUDGET} epochs
(batch {bs})", fontsize=8.5); ax.set_xlabel("learning rate")
        for a in range(4):
            for b in range(3):
                e = g[(g.opt == Q.OPTS[a]) & (g.lr == Q.LRS[b]) & (g.bs ==
bs)].ep_target_median.iloc[0]
                ax.text(b, a, f"{mat[a, b]:.2f}\n(median {'-' if not np.isfinite(e) else
int(e)} ep)", ha="center", va="center", fontsize=8)
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q4_budget_heatmap.png"), dpi=200);
plt.close()
    log("done")
```

q5_cnn_design.py – Q5

```
"""Q5 - Derive a convolutional network from a design principle (PCB-defect motivation).
Stand-in data: Fashion-MNIST (28x28, 10 classes); no PCB images are reachable here.  Train
20,000 / validation 5,000 (disjoint) / test 10,000.

Models (all trained on UN-shifted images, Adam lr 1e-3, batch 128, 10 epochs, 3 seeds):
  FC      : 784-256-128-10 (ReLU)                                    - fully connected on
flattened image
  CNN     : conv3x3(1->16,p1)-ReLU-pool2 - conv3x3(16->32,p1)-ReLU-pool2 - flatten(1568) -
FC(10)
  CNN-GAP : conv3x3(1->16)-pool - conv3x3(16->32)-pool - conv3x3(32->64) - global average pool
- FC(64->10)
Translation test: test images shifted by k pixels (left/right/up/down, zero background),
accuracy averaged over the 4 directions.
```

```
"""
import sys, os, json, time
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C

BGV = (0 - C.FM_MEAN) / C.FM_STD
SHIFTS = [0, 1, 2, 3, 4, 6]



class FC(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(), nn.Linear(256,
128), nn.ReLU(), nn.Linear(128, 10))
    def forward(self, x): return self.net(x)



class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1, self.c2 = nn.Conv2d(1, 16, 3, padding=1), nn.Conv2d(16, 32, 3, padding=1)
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x, maps=False):
        a1 = F.relu(self.c1(x)); p1 = F.max_pool2d(a1, 2); a2 = F.relu(self.c2(p1)); p2 =
F.max_pool2d(a2, 2)
        o = self.fc(p2.flatten(1))
        return (o, a1, a2) if maps else o



class CNNGAP(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1, self.c2, self.c3 = nn.Conv2d(1, 16, 3, padding=1), nn.Conv2d(16, 32, 3,
padding=1), nn.Conv2d(32, 64, 3, padding=1)
        self.fc = nn.Linear(64, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.c1(x)), 2); x = F.max_pool2d(F.relu(self.c2(x)), 2); x =
F.relu(self.c3(x))
        return self.fc(x.mean((2, 3)))



MODELS = {"FC": FC, "CNN": CNN, "CNN-GAP": CNNGAP}



def shift(x, dy, dx):
    out = torch.full_like(x, BGV)
    H, W = x.shape[-2:]
    ys, yd = (slice(0, H - dy), slice(dy, H)) if dy >= 0 else (slice(-dy, H), slice(0, H + dy))
    xs, xd = (slice(0, W - dx), slice(dx, W)) if dx >= 0 else (slice(-dx, W), slice(0, W + dx))
    out[..., yd, xd] = x[..., ys, xs]
    return out



@torch.no_grad()
def acc(m, X, y, bs=1000):
    m.eval(); c = 0
    for i in range(0, len(X), bs):
        c += (m(X[i:i + bs]).argmax(1) == y[i:i + bs]).sum().item()
    return c / len(X)



def count_macs(name):
    # analytical MACs per image (conv: k*k*Cin*Cout*H*W; linear: in*out)
    if name == "FC": return 784 * 256 + 256 * 128 + 128 * 10
```

```
    if name == "CNN": return 9 * 1 * 16 * 28 * 28 + 9 * 16 * 32 * 14 * 14 + 1568 * 10
    return 9 * 1 * 16 * 28 * 28 + 9 * 16 * 32 * 14 * 14 + 9 * 32 * 64 * 7 * 7 + 64 * 10



if __name__ == "__main__":
    log = C.Logger("q5")
    Xtr, ytr, Xva, yva, Xte, yte = C.fmnist_split(20000, 5000, seed=0)
    log(f"Fashion-MNIST: train {tuple(Xtr.shape)} val {tuple(Xva.shape)} test
{tuple(Xte.shape)}")
    rows, shift_rows, store = [], [], {}
    for name, cls in MODELS.items():
        for seed in C.SEEDS:
            C.set_seed(seed); m = cls(); opt = torch.optim.Adam(m.parameters(), lr=1e-3); gen =
torch.Generator().manual_seed(seed)
            t0 = time.perf_counter()
            for ep in range(10):
                m.train()
                for xb, yb in C.batches(Xtr, ytr, 128, gen):
                    opt.zero_grad(); F.cross_entropy(m(xb), yb).backward(); opt.step()
            tt = time.perf_counter() - t0
            va, te = acc(m, Xva, yva), acc(m, Xte, yte)
            rows.append(dict(model=name, seed=seed, params=sum(p.numel() for p in
m.parameters()), macs=count_macs(name), train_secs=tt, val_acc=va, test_acc=te))
            for k in SHIFTS:
                a = np.mean([acc(m, shift(Xte, dy, dx), yte) for dy, dx in [(0, k), (0, -k),
(k, 0), (-k, 0)]])
                shift_rows.append(dict(model=name, seed=seed, shift=k, acc=float(a)))
            log(f"{name:8s} seed={seed} params={rows[-1]['params']} train={tt:.1f}s
val={va:.4f} test={te:.4f} "
                + " ".join(f"s{k}={shift_rows[-len(SHIFTS)+i]['acc']:.3f}" for i, k in
enumerate(SHIFTS)))
            if seed == 0:
                store[name] = m
    df = pd.DataFrame(rows); df.to_csv(os.path.join(C.RES, "q5_models.csv"), index=False)
    sd = pd.DataFrame(shift_rows); sd.to_csv(os.path.join(C.RES, "q5_translation.csv"),
index=False)
    log("\n" + df.groupby("model",
sort=False).mean(numeric_only=True).drop(columns="seed").round(4).to_string())
    piv = sd.groupby(["model", "shift"]).acc.mean().unstack(); log("\ntranslation test (mean
acc):\n" + piv.round(4).to_string())

    # ---- equivariance check on the first conv layer: conv(shift(x)) == shift(conv(x)) away
from the border
    cnn = store["CNN"]; x = Xte[:64]
    with torch.no_grad():
        ref = F.conv2d(x, cnn.c1.weight, padding=1); sh = F.conv2d(shift(x, 0, 3),
cnn.c1.weight, padding=1)
        # interior region only (rows 4..23, columns 6..23): away from padding/fill effects the
output must be the same map moved 3 px
        err = (sh[..., 4:24, 6:24] - torch.roll(ref, 3, 3)[..., 4:24, 6:24]).abs().max().item()
    log(f"equivariance check (first conv, bias-free, 3-px shift, interior): max |conv(shift x)
- shift(conv x)| = {err:.2e}")
    json.dump(dict(equivariance_max_abs_err=err), open(os.path.join(C.RES,
"q5_equivariance.json"), "w"))

    # ---- figures: translation curves
    plt.figure(figsize=(5.8, 3.8))
    for name, c in zip(MODELS, ["#d1495b", "#2e86ab", "#3a9d5d"]):
        g = sd[sd.model == name].groupby("shift").acc; plt.errorbar(SHIFTS, g.mean(), g.std(),
marker="o", color=c, label=name, capsize=2)
    plt.xlabel("shift of test images (pixels)"); plt.ylabel("test accuracy");
plt.grid(alpha=.3); plt.legend(); plt.tight_layout()
    plt.savefig(os.path.join(C.FIG, "q5_translation_test.png"), dpi=200); plt.close()

    # ---- first-layer filters and feature maps
    W = cnn.c1.weight.detach()[:, 0].numpy()
    fig, axs = plt.subplots(2, 8, figsize=(10.5, 3.0))
    for i, ax in enumerate(axs.flat):
        v = np.abs(W[i]).max(); ax.imshow(W[i], cmap="RdBu_r", vmin=-v, vmax=v);
ax.set_xticks([]); ax.set_yticks([]); ax.set_title(f"f{i}", fontsize=8)
```

```
    plt.suptitle("CNN first-layer 3x3 filters (16)", fontsize=10); plt.tight_layout();
plt.savefig(os.path.join(C.FIG, "q5_filters.png"), dpi=200); plt.close()
    fcw = store["FC"].net[1].weight.detach().reshape(256, 28, 28).numpy()
    fig, axs = plt.subplots(1, 8, figsize=(10.5, 1.6))
    for i, ax in enumerate(axs.flat):
        v = np.abs(fcw[i]).max(); ax.imshow(fcw[i], cmap="RdBu_r", vmin=-v, vmax=v);
ax.set_xticks([]); ax.set_yticks([]); ax.set_title(f"unit {i}", fontsize=8)
    plt.suptitle("FC first-layer weights of 8 hidden units (each is a full 28x28 template)",
fontsize=9); plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q5_fc_weights.png"),
dpi=200); plt.close()
    with torch.no_grad():
        img = Xte[3:4]; _, a1, a2 = cnn(img, maps=True)
    fig, axs = plt.subplots(3, 8, figsize=(10.5, 4.2))
    axs[0, 0].imshow(img[0, 0], cmap="gray"); axs[0, 0].set_title("input", fontsize=8)
    for i in range(1, 8): axs[0, i].axis("off")
    for i in range(8):
        axs[1, i].imshow(a1[0, i], cmap="viridis"); axs[1, i].set_title(f"conv1 map {i}",
fontsize=7)
        axs[2, i].imshow(a2[0, i], cmap="viridis"); axs[2, i].set_title(f"conv2 map {i}",
fontsize=7)
    for a in axs.flat: a.set_xticks([]); a.set_yticks([])
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q5_feature_maps.png"), dpi=200);
plt.close()
    log("done")
```

q6_architecture.py – Q6

```
"""Q6 - Architecture design: capacity, cost and the deployment boundary (on-device wildlife
camera trap).
Stand-in data: Fashion-MNIST (10 classes).  Train 10,000 / validation 5,000 (disjoint) / test
10,000; resolution 28 or 14 (2x2 average pooling).

Architecture family  Net(d, w, r)   (own design, not VGG/ResNet), d >= 2 convolution layers,
width w, input resolution r (multiple of 4):
   conv3x3(1->w, pad 1)+ReLU  at r x r      -> maxpool2        (layer 1)
   conv3x3(w->w, pad 1)+ReLU  at r/2        -> maxpool2        (layer 2)
   (d-2) x [conv3x3(w->w, pad 1)+ReLU] at r/4                  (layers 3..d)
   global average pool -> Linear(w -> 10)
Analytical formulas (per image):
   params = (9*1*w + w) + (d-1)*(9*w*w + w) + (10*w + 10)
   MACs   = 9*w*r^2 + 9*w^2*(r/2)^2 + (d-2)*9*w^2*(r/4)^2 + 10*w          (bias additions not
counted)
   receptive field (input pixels) at the last conv layer + pool: 8*d - 6 (d=2: conv2 sees 8,
the pool after it widens it to 10)
Training: Adam lr 3e-3, batch 128, 10 epochs, 3 seeds.  Resolutions 16 and 32 are obtained by
resizing the 28x28 images (bilinear).
Budget (chosen in advance): model file <= 64 KB (float32 weights, ~1/4 of a 256 KB MCU flash)
and <= 1.0 M MACs per image
(~20 ms on a Cortex-M7-class core sustaining ~50 M float MAC/s).
"""
import sys, os, io, json, time, math
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C

ARCHS = [(2, 4, 16), (2, 8, 16), (4, 8, 16), (2, 16, 16), (2, 8, 32), (4, 16, 32), (4, 32, 32)]
# (depth, width, resolution)
NAMES = {(2, 4, 16): "tiny", (2, 8, 16): "base", (4, 8, 16): "depth x2", (2, 16, 16): "width
x2", (2, 8, 32): "resolution x2",
         (4, 16, 32): "all x2", (4, 32, 32): "large"}
BUDGET_KB, BUDGET_MACS = 64.0, 1.0e6



class Net(nn.Module):
    def __init__(self, d, w, r):
        super().__init__()
```

```
        self.d, self.w, self.r = d, w, r
        assert d >= 2 and r % 4 == 0
        self.c1 = nn.Conv2d(1, w, 3, padding=1)
        self.cs = nn.ModuleList([nn.Conv2d(w, w, 3, padding=1) for _ in range(d - 1)])
        self.fc = nn.Linear(w, 10)

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.c1(x)), 2)
        x = F.max_pool2d(F.relu(self.cs[0](x)), 2)
        for c in self.cs[1:]:
            x = F.relu(c(x))
        return self.fc(x.mean((2, 3)))



def params_formula(d, w, r): return (9 * w + w) + (d - 1) * (9 * w * w + w) + (10 * w + 10)
def macs_formula(d, w, r): return 9 * w * r * r + 9 * w * w * (r // 2) ** 2 + (d - 2) * 9 * w *
w * (r // 4) ** 2 + 10 * w
def rf_formula(d): return 8 * d - 6



def macs_by_hooks(m, r):
    """independent MAC counter: forward hooks on every conv / linear layer"""
    tot = [0]
    def hk(mod, inp, out):
        if isinstance(mod, nn.Conv2d):
            tot[0] += out.shape[2] * out.shape[3] * mod.out_channels * mod.in_channels *
mod.kernel_size[0] * mod.kernel_size[1]
        else:
            tot[0] += mod.in_features * mod.out_features
    hs = [l.register_forward_hook(hk) for l in m.modules() if isinstance(l, (nn.Conv2d,
nn.Linear))]
    with torch.no_grad(): m(torch.zeros(1, 1, r, r))
    for h in hs: h.remove()
    return tot[0]



def receptive_field(m):
    """recurrence RF_l = RF_{l-1} + (k-1)*jump_{l-1};  jump_l = jump_{l-1}*stride"""
    rf, j = 1, 1; seq = []
    rf += 2 * j; seq.append(("conv1", rf, j)); rf += 1 * j; j *= 2; seq.append(("pool1", rf,
j))
    rf += 2 * j; seq.append(("conv2", rf, j)); rf += 1 * j; j *= 2; seq.append(("pool2", rf,
j))
    for i in range(m.d - 2):
        rf += 2 * j; seq.append((f"conv{i+3}", rf, j))
    return seq



def hand_table(d, w, r):
    """layer-by-layer hand calculation (the report reproduces this table)"""
    rows = [("input", (1, r, r), 0, 0)]; h = r
    rows.append(("conv1 3x3", (w, h, h), 9 * 1 * w + w, 9 * 1 * w * h * h)); h //= 2
    rows.append(("maxpool 2x2", (w, h, h), 0, 0))
    rows.append(("conv2 3x3", (w, h, h), 9 * w * w + w, 9 * w * w * h * h)); h //= 2
    rows.append(("maxpool 2x2", (w, h, h), 0, 0))
    for i in range(d - 2):
        rows.append((f"conv{i+3} 3x3", (w, h, h), 9 * w * w + w, 9 * w * w * h * h))
    rows.append(("global avg pool", (w,), 0, 0)); rows.append(("linear", (10,), 10 * w + 10, 10
* w))
    return rows



def inference_ms(m, r, n=300):
    m.eval(); x = torch.randn(1, 1, r, r)
    with torch.no_grad():
        for _ in range(30): m(x)
        ts = []
        for _ in range(n):
            t = time.perf_counter(); m(x); ts.append(time.perf_counter() - t)
```

```
    return float(np.median(ts) * 1e3)



@torch.no_grad()
def acc(m, X, y, bs=1000):
    m.eval(); c = 0
    for i in range(0, len(X), bs): c += (m(X[i:i + bs]).argmax(1) == y[i:i + bs]).sum().item()
    return c / len(X)



if __name__ == "__main__":
    log = C.Logger("q6")
    # ------------------------------------------------------------- hand verification of the small architecture
    d, w, r = 2, 8, 16
    rows = hand_table(d, w, r)
    tp, tm = sum(x[2] for x in rows), sum(x[3] for x in rows)
    log("Hand calculation for base architecture d=2,w=8,r=16:")
    for n, s, p, mc in rows: log(f"  {n:16s} out={str(s):14s} params={p:6d} MACs={mc:8d}")
    log(f"  TOTAL params={tp}  MACs={tm}")
    m = Net(d, w, r); fw = sum(p.numel() for p in m.parameters())
    assert fw == tp == params_formula(d, w, r), (fw, tp)
    assert macs_by_hooks(m, r) == tm == macs_formula(d, w, r)
    try:
        from torchinfo import summary
        s = summary(m, input_size=(1, 1, r, r), col_names=["input_size", "output_size", "num_params", "mult_adds"], verbose=0)
        open(os.path.join(C.RES, "q6_torchinfo_summary.txt"), "w").write(str(s));
log("torchinfo summary saved; total params", s.total_params, "mult-adds", s.total_mult_adds)
        ti = dict(total_params=int(s.total_params), total_mult_adds=int(s.total_mult_adds))
    except Exception as e:
        ti = dict(error=str(e)); log("torchinfo failed:", e)
    json.dump(dict(hand_rows=[[a, list(b), c, e] for a, b, c, e in rows], hand_params=tp, hand_macs=tm, framework_params=fw, hook_macs=macs_by_hooks(m, r), torchinfo=ti,
                   rf=receptive_field(m)), open(os.path.join(C.RES, "q6_hand_check.json"), "w"), indent=1)
    log("receptive-field table (layer, RF, jump):", receptive_field(m))

    # ------------------------------------------------------------- growth law check: double each dimension from the base
    g = []
    for lab, (d2, w2, r2) in [("depth x2", (4, 8, 16)), ("width x2", (2, 16, 16)), ("resolution x2", (2, 8, 32))]:
        g.append(dict(change=lab, params_ratio=params_formula(d2, w2, r2) / params_formula(2, 8, 16), macs_ratio=macs_formula(d2, w2, r2) / macs_formula(2, 8, 16)))
    # asymptotic ratios for a deep/wide base (d=8,w=32,r=28) to show the pure scaling laws
    for lab, (d2, w2, r2) in [("depth x2", (16, 32, 32)), ("width x2", (8, 64, 32)), ("resolution x2", (8, 32, 64))]:
        g.append(dict(change=lab + " (from d=8,w=32,r=32)", params_ratio=params_formula(d2, w2, r2) / params_formula(8, 32, 32), macs_ratio=macs_formula(d2, w2, r2) / macs_formula(8, 32, 32)))
    pd.DataFrame(g).to_csv(os.path.join(C.RES, "q6_growth_formula.csv"), index=False); log("\n" + pd.DataFrame(g).round(3).to_string())

    # ------------------------------------------------------------- experiments
    Xtr, ytr, Xva, yva, Xte, yte = C.fmnist_split(10000, 5000, seed=0)
    rs = lambda X, r: F.interpolate(X, size=(r, r), mode="bilinear", align_corners=False)
    out = []
    for (d, w, r) in ARCHS:
        D = (rs(Xtr, r), rs(Xva, r), rs(Xte, r))
        for seed in C.SEEDS:
            C.set_seed(seed); m = Net(d, w, r); opt = torch.optim.Adam(m.parameters(), lr=3e-3); gen = torch.Generator().manual_seed(seed)
            t0 = time.perf_counter()
            for ep in range(10):
                m.train()
                for xb, yb in C.batches(D[0], ytr, 128, gen):
                    opt.zero_grad(); F.cross_entropy(m(xb), yb).backward(); opt.step()
            tt = time.perf_counter() - t0
            buf = io.BytesIO(); torch.save(m.state_dict(), buf)
```

```
            res = dict(name=NAMES[(d, w, r)], depth=d, width=w, res=r, seed=seed,
params=sum(p.numel() for p in m.parameters()), macs=macs_by_hooks(m, r),
                       macs_formula=macs_formula(d, w, r), params_formula=params_formula(d, w,
r), rf=rf_formula(d), file_kb=buf.tell() / 1024,
                       train_secs=tt, val_acc=acc(m, D[1], yva), test_acc=acc(m, D[2], yte))
            res["infer_ms"] = inference_ms(m, r) if seed == 0 else np.nan
            out.append(res)
            log(f"{res['name']:14s} (d={d},w={w},r={r}) seed={seed} params={res['params']}
MACs={res['macs']} file={res['file_kb']:.1f}KB "
                f"val={res['val_acc']:.4f} test={res['test_acc']:.4f}
infer={res['infer_ms']:.3f}ms train={tt:.0f}s")
    df = pd.DataFrame(out); df.to_csv(os.path.join(C.RES, "q6_runs.csv"), index=False)
    assert (df.macs == df.macs_formula).all() and (df.params == df.params_formula).all()
    s = df.groupby(["name", "depth", "width", "res"], sort=False).agg(params=("params",
"first"), macs=("macs", "first"), file_kb=("file_kb", "first"), rf=("rf", "first"),
        infer_ms=("infer_ms", "mean"), val_acc=("val_acc", "mean"), val_std=("val_acc", "std"),
test_acc=("test_acc", "mean"), train_secs=("train_secs", "mean")).reset_index()
    s["within_budget"] = (s.file_kb <= BUDGET_KB) & (s.macs <= BUDGET_MACS)
    s.to_csv(os.path.join(C.RES, "q6_summary.csv"), index=False); log("\n" +
s.round(4).to_string())
    ok = s[s.within_budget].sort_values("val_acc", ascending=False)
    best = ok.iloc[0]; big = s.sort_values("val_acc", ascending=False).iloc[0]
    dec = dict(budget_kb=BUDGET_KB, budget_macs=BUDGET_MACS, chosen=best["name"],
chosen_val_acc=float(best.val_acc), chosen_test_acc=float(best.test_acc),
               largest=big["name"], largest_val_acc=float(big.val_acc),
largest_test_acc=float(big.test_acc),
               sacrificed_val=float(big.val_acc - best.val_acc),
sacrificed_test=float(big.test_acc - best.test_acc))
    C.save_json(dec, "q6_deployment_decision.json"); log("DECISION:", dec)

    # ------------------------------------------------------------------ figures
    fig, axs = plt.subplots(1, 3, figsize=(14, 3.9))
    gg = pd.DataFrame(g[:3]); x = np.arange(3); wd = 0.36
    axs[0].bar(x - wd / 2, gg.params_ratio, wd, label="parameters", color="#2e86ab");
axs[0].bar(x + wd / 2, gg.macs_ratio, wd, label="MACs", color="#e09f3e")
    axs[0].set_xticks(x); axs[0].set_xticklabels(gg.change); axs[0].set_ylabel("growth factor
vs base (d=2,w=8,r=16)"); axs[0].legend(); axs[0].grid(alpha=.3, axis="y")
    for a, b in [(axs[1], "macs"), (axs[2], "params")]:
        for _, rr in s.iterrows():
            a.scatter(rr[b], rr.val_acc, s=40, color="#3a9d5d" if rr.within_budget else
"#d1495b"); a.annotate(rr["name"], (rr[b], rr.val_acc), fontsize=7, xytext=(3, -9),
textcoords="offset points")
        a.set_xscale("log"); a.set_ylabel("validation accuracy"); a.grid(alpha=.3,
which="both")
    axs[1].axvline(BUDGET_MACS, color="k", ls="--", lw=.9); axs[1].set_xlabel("MACs per
image"); axs[2].set_xlabel("parameters")
    axs[2].axvline(BUDGET_KB * 1024 / 4, color="k", ls="--", lw=.9)
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q6_growth_and_tradeoff.png"),
dpi=200); plt.close()
    log("done")
```

q6_infer_timing.py – Q6 latency re-measurement

```
"""Re-measures batch-1 CPU inference latency of the Q6 architectures on an otherwise idle
machine
(untrained weights - latency does not depend on the weight values) and updates
results/q6_summary.csv."""
import os, sys, numpy as np, pandas as pd, torch
sys.path.insert(0, os.path.dirname(__file__))
import common as C
from q6_architecture import Net, ARCHS, NAMES, inference_ms
torch.set_num_threads(1)
path = os.path.join(C.RES, "q6_summary.csv"); s = pd.read_csv(path)
lat = {}
for a in ARCHS:
    torch.manual_seed(0); m = Net(*a)
    runs = [inference_ms(m, a[2], n=500) for _ in range(5)]
    lat[NAMES[a]] = float(np.median(runs)); print(NAMES[a], a, "median of 5 x 500 passes: %.4f
ms (min %.4f, max %.4f)" % (lat[NAMES[a]], min(runs), max(runs)))
s["infer_ms_loaded"] = s["infer_ms"]; s["infer_ms"] = s["name"].map(lat)
```

```
s.to_csv(path, index=False); C.save_json(lat, "q6_latency_idle.json")
```

q7_redesign.py – Q7

```
"""Q7 - Diagnose, redesign and validate.
Model selected from Q1: 12-hidden-layer sigmoid MLP (Xavier init) for credit-card fraud
screening - vanishing gradients.
Re-uses the Q1 pipeline (same data, same seeds 0-2, same 15-epoch SGD+momentum budget).
"""
import sys, os, json, time
sys.path.insert(0, os.path.dirname(__file__))
import numpy as np, pandas as pd, torch, torch.nn.functional as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import common as C
import q1_activations as Q

DESIGNS = [  # name, act, init, residual
    ("Original (sigmoid, Xavier)", "sigmoid", "xavier", False),
    ("R1 sigmoid + gain-corrected init", "sigmoid", "xavier4", False),
    ("R2 ReLU + He", "relu", "he", False),
    ("R3 sigmoid + skip", "sigmoid", "xavier", True),
    ("R4 ReLU + He + skip (final)", "relu", "he", True),
]
DEPTH = 12

if __name__ == "__main__":
    log = C.Logger("q7")
    data = Q.get_data()
    (Xtr, ytr) = data[0]

    # ------------------------------------------------------------------ A. diagnosis from the
Q1 evidence
    raw = json.load(open(os.path.join(C.RES, "q1_runs_raw.json")))
    orig = [r for r in raw if r["tag"] == "deep" and r["act"] == "sigmoid"]
    g_early = np.mean([r["g_early"] for r in orig], 0)
    g_init = np.mean([r["g_init"] for r in orig], 0)
    dphi = np.mean([r["dphi_init"] for r in orig], 0)
    decay = np.exp(np.diff(np.log(g_early[::-1]))).tolist()   # ratio g_{l-1}/g_l moving
towards the input
    diag = dict(g_early=g_early.tolist(), g_init=g_init.tolist(), dphi_init=dphi.tolist(),
                ratio_first_over_last=float(g_early[0] / g_early[-1]),
                per_layer_decay_geomean=float(np.exp(np.mean(np.log(decay[:-1])))),
                loss_curve=np.mean([[c[1] for c in r["curve"]] for r in orig], 0).tolist(),
                val_rec_curve=np.mean([[c[4] for c in r["curve"]] for r in orig], 0).tolist())
    C.save_json(diag, "q7_diagnosis.json")
    log(f"Diagnosis: |g1|/|g_out| = {diag['ratio_first_over_last']:.2e}; per-layer decay (geo-
mean) = {diag['per_layer_decay_geomean']:.3f}; "
        f"mean sigmoid' at init = {dphi.mean():.3f}; train loss
epoch1={diag['loss_curve'][0]:.4f} epoch15={diag['loss_curve'][-1]:.4f}")

    # ------------------------------------------------------------------ B. run the designs
    runs = []
    for name, act, init, res in DESIGNS:
        for seed in C.SEEDS:
            r = Q.run_one(act, DEPTH, init, seed, data, res=res, tag=name)
            runs.append(r)
            log(f"{name:36s} seed={seed} train_loss={r['train_loss']:.4f}
val_rec={r['val_rec']:.3f} val_prec={r['val_prec']:.3f} "
                f"auprc={r['val_auprc']:.3f} g1/gout(early)={r['g_early'][0]/r['g_early'][-
1]:.2e} dead={np.nanmean(r['dead_final']) if act=='relu' else float('nan'):.3f}")
    json.dump(runs, open(os.path.join(C.RES, "q7_runs_raw.json"), "w"))

    # ------------------------------------------------------------------ C. timing micro-
benchmark (100 SGD steps, batch 256)
    def bench(act, init, res, reps=5):
        C.set_seed(0); m = Q.MLP(act, DEPTH, init, res=res); opt =
torch.optim.SGD(m.parameters(), lr=0.01, momentum=0.9)
```

```
        xb, yb = Xtr[:256], ytr[:256]; ts = []
        for _ in range(reps):
            t = time.perf_counter()
            for _ in range(100):
                opt.zero_grad(); F.binary_cross_entropy_with_logits(m(xb), yb).backward();
opt.step()
            ts.append((time.perf_counter() - t) / 100 * 1e3)
        return float(np.median(ts))
    tms = {n: bench(a, i, r) for n, a, i, r in DESIGNS}
    log("ms/step: " + ", ".join(f"{k[:2]}={v:.2f}" for k, v in tms.items()))

    rows = []
    for name, act, init, res in DESIGNS:
        rs = [r for r in runs if r["tag"] == name]
        ratio = [r["g_early"][0] / r["g_early"][-1] for r in rs]
        rows.append(dict(design=name, params=rs[0]["n_params"],
grad_ratio_early=float(np.exp(np.mean(np.log(ratio)))),
                         g1_early=float(np.mean([r["g_early"][0] for r in rs])),
                         train_loss=np.mean([r["train_loss"] for r in rs]),
val_loss=np.mean([r["val_loss"] for r in rs]),
                         val_acc=np.mean([r["val_acc"] for r in rs]),
val_prec=np.mean([r["val_prec"] for r in rs]),
                         val_rec=np.mean([r["val_rec"] for r in rs]),
val_auprc=np.mean([r["val_auprc"] for r in rs]),
                         test_rec=np.mean([r["test_rec"] for r in rs]),
test_prec=np.mean([r["test_prec"] for r in rs]),
                         val_rec_std=np.std([r["val_rec"] for r in rs]),
val_auprc_std=np.std([r["val_auprc"] for r in rs]),
                         dead_relu_final=float(np.mean([np.nanmean(r["dead_final"]) for r in
rs])) if act == "relu" else np.nan,
                         ms_per_step=tms[name], diverged=int(sum(r["diverged"] for r in rs))))
    tab = pd.DataFrame(rows); tab.to_csv(os.path.join(C.RES, "q7_comparison.csv"), index=False)
    log("\n" + tab.round(4).to_string())

    # ------------------------------------------------------------------ D. depth stress test
at init
    sw = []
    for name, act, init, res in [DESIGNS[0], DESIGNS[2], DESIGNS[4]]:
        for depth in [4, 8, 12, 16, 24, 32, 48, 64]:
            rs = []
            for s in range(5):
                C.set_seed(100 + s); m = Q.MLP(act, depth, init, res=res)
                g, _ = Q.init_gradient(m, Xtr[:2048], ytr[:2048]); rs.append(g[0] / g[-1])
            sw.append(dict(design=name, depth=depth, ratio=float(np.exp(np.mean(np.log(rs))))))
    sw = pd.DataFrame(sw); sw.to_csv(os.path.join(C.RES, "q7_depth_stress.csv"), index=False)
    log("\n" + sw.pivot(index="depth", columns="design",
values="ratio").to_string(float_format=lambda v: f"{v:.2e}"))

    # ------------------------------------------------------------------ figures
    cols = ["#d1495b", "#e09f3e", "#2e86ab", "#8e6bbf", "#3a9d5d"]
    fig, axs = plt.subplots(1, 3, figsize=(14.5, 3.9))
    for (name, act, init, res), c in zip(DESIGNS, cols):
        g = np.mean([r["g_early"] for r in runs if r["tag"] == name], 0)
        axs[0].plot(range(1, len(g) + 1), g, "o-", color=c, ms=3.5, label=name)
        cur = np.mean([[x[1] for x in r["curve"]] for r in runs if r["tag"] == name], 0)
        axs[1].plot(range(1, len(cur) + 1), cur, color=c, label=name)
    axs[0].set_yscale("log"); axs[0].set_xlabel("layer (1 = first hidden ... 13 = output)");
axs[0].set_ylabel("||dL/dW|| (first 50 steps)"); axs[0].grid(alpha=.3, which="both")
    axs[1].set_xlabel("epoch"); axs[1].set_ylabel("training loss"); axs[1].grid(alpha=.3)
    axs[0].legend(fontsize=6.5)
    for name, c in zip([DESIGNS[0][0], DESIGNS[2][0], DESIGNS[4][0]], [cols[0], cols[2],
cols[4]]):
        s = sw[sw.design == name]; axs[2].plot(s.depth, s.ratio, "o-", color=c,
label=name.split(" (")[0])
    axs[2].set_yscale("log"); axs[2].set_xlabel("hidden layers"); axs[2].set_ylabel("|g layer1|
/ |g output| at init"); axs[2].grid(alpha=.3, which="both"); axs[2].legend(fontsize=7)
    plt.tight_layout(); plt.savefig(os.path.join(C.FIG, "q7_redesign_comparison.png"),
dpi=200); plt.close()
    log("done")
```